Implementation and Evaluation of the 1990 Simplified Jordan Sorting Algorithm

Master's Thesis

Yixue Wang

Review draft

2 November 2026

Freie Universität Berlin
Department of Mathematics and Computer Science
Institute of Computer Science
Theoretical Computer Science

For supervisor review

Current implementation and
paired experiments

Abstract

Jordan sorting asks for the order of the intersection points of a simple curve and a line along the line, given their order along the curve. Hoffmann et al. (1986) and Fung et al. (1990) showed that this can be done in linear time when the underlying dynamic lists support the required finger-search operations and cost bounds. This thesis presents an executable Python reconstruction of the 1990 simplified algorithm. A single sorting procedure runs on two interchangeable backends: ordinary Python lists and a heterogeneous red–black finger tree. For the tree backend, a manual amortized analysis gives linear total work on valid inputs under unit-cost comparison and machine-word assumptions, with expected dictionary lookup and amortized growth.

In a paired experiment on 140 controlled inputs of length 32–513 and twelve additional inputs exercising two-sided splits, both backends produced correct output in all 66,560 measured calls. Nevertheless, the tree backend had a higher median runtime on every input. The median tree/list ratio was about 2.9 on the common set (range 1.1–3.5) and 3.2 on the split set. A supplementary diagnostic examined instrumentation hooks, the code that intercepts field accesses for work counting. Although counting was disabled in the measured configuration, removing these hooks reduced median tree runtimes by about 38–52% and made the tree faster on four of fifteen inputs. For the measured Python configuration, the tree's stronger structural bound thus coexists with higher runtime over the tested inputs.

Contents

Abstract . . . . . **i**

List of Figures . . . . . **vii**

List of Tables . . . . . **ix**

1. Introduction **1**
1.1. Motivation and Scope 1
1.2. Research Questions 1
1.3. Contributions and Findings 2
1.4. Reading Guide 2

2. Background **3**
2.1. Jordan Sequences and the Sorting Problem 3
2.2. The Structural Restriction Behind Jordan Sorting 4
2.3. The 1986 Linear-Time Algorithm 5
2.4. The 1990 Simplification 6
2.5. Related Work and Positioning 6
2.5.1. Ordered Lists and Finger Search 6
2.5.2. Jordan Sorting and Geometric Applications 7
2.6. Position and Scope of This Thesis 7

3. Algorithm Reconstruction **9**
3.1. From the Paper to the Implementation 9
3.2. Pairs, Intervals, and the Two Families 10
3.2.1. Families and Sibling Lists 10
3.3. Maintained State and Prefix Invariant 11
3.4. Initialization 12
3.5. Boundary Selection 12
3.5.1. Step 1: Predecessor-Side Boundary 12
3.5.2. Step 2: Successor-Side Boundary 12

3.6. Step 3: Increasing Orientation . . . 13
3.6.1. Step 3(a): Insert the New Pair . . . 13
3.6.2. Step 3(b): Split Off Newly Enclosed Children . . . 13
3.6.3. Step 3(c): Insert the New Point . . . 13
3.7. Step 3: Decreasing Orientation . . . 13
3.8. Executable Anchor Clarifications . . . 14
3.8.1. Curve Endpoint Versus Geometric Endpoint . . . 14
3.8.2. Odd-Index z_1 Output Anchor . . . 15
3.9. Ownership Transfer and Failure Semantics . . . 15
3.10. End-to-End Procedure . . . 17
3.11. Correctness and Operation Costs . . . 19

4. Implementation . . . 21
4.1. Architecture and Versions . . . 21
4.2. Data Representation . . . 21
4.2.1. Partial Sorted Order . . . 22
4.2.2. Sibling Lists and Ownership . . . 22
4.3. Core–Backend Interface and Update Semantics . . . 22
4.3.1. Initialization, Boundaries and Stage State . . . 22
4.3.2. Pair Insertion and Failure Propagation . . . 23
4.3.3. Split Plans and Ownership Transfer . . . 23
4.3.4. Point Insertion and Completion . . . 24
4.4. Certification, Diagnostics and Replay . . . 24
4.4.1. Correctness Coverage . . . 25
4.5. Execution Policies and Observation Costs . . . 26
4.6. Operation Costs and Complexity . . . 27

5. Experimental Method . . . 29
5.1. Study Design and Versions . . . 29
5.2. Current Inputs and Qualification . . . 29
5.3. Compared Implementations and Call Scopes . . . 30
5.4. Timing, Calibration and Scheduling . . . 30
5.4.1. Scheduling and Noise . . . 30
5.4.2. Recorded Environment . . . 31
5.5. Aggregation and Statistical Units . . . 31
5.6. Experimental Records and Validation . . . 31
5.6.1. Fail-Closed Validation . . . 31
5.7. Study Stages . . . 32

5.8. Historical Study: Separate Scope . . . 32

6. Results . . . 33
6.1. Execution Summary and Output Checks . . . 33
6.2. Common-Set Runtime and Paired Ratios . . . 33
6.2.1. Structural Strata . . . 35
6.3. Targeted Split Set . . . 35
6.4. Variability and Measurement Accounting . . . 36
6.5. Supplementary Instrumentation-Hook Diagnostic . . . 36
6.6. Historical Rollback-Enabled Evidence . . . 38
6.7. What the Comparison Shows . . . 38

7. Limitations . . . 39
7.1. Scope and Version . . . 39
7.2. Mathematical Model and Backend Fidelity . . . 39
7.3. Input Coverage and Scale . . . 40
7.4. Timing Scopes and Environment . . . 40
7.5. Statistical and Exploratory Reach . . . 41
7.6. Output Provenance and Recognition . . . 41
7.7. Consequences for the Conclusions . . . 42

8. Conclusion . . . 43
8.1. Answers to the Research Questions . . . 43
8.2. Contributions and Further Work . . . 44

A. Representation, Locality and Cost Argument . . . 45
A.1. Decoding and Preservation . . . 45
A.2. Locality and Exact Grouping . . . 48
A.3. Costs of Mixed Operations . . . 50

B. Evidence and AI Use . . . 55
B.1. Evidence and Source Index . . . 55
B.2. Documentation of AI Use . . . 55
B.3. Reflection on AI Use . . . 56

Bibliography . . . 59

List of Figures

2.1. Jordan sorting from curve order to axis order. 4

3.1. Step 3(c) output-anchor correction for z_1. 16
3.2. Running example of the ordinary-list reconstruction. 18

4.1. Pair families and sibling-list ownership. 23

5.1. Current evidence flow. Qualification, calibration and formal sampling have separate identities. The same predetermined inputs and per-input B reach the samples; independent reconstruction checks each archive. 32

6.1. Current common set. Left: medians of input medians, with a logarithmic time axis. Right: all fourteen input ratios per length; horizontal marks are their medians. Horizontal offsets separate points. The dashed line denotes equal runtime. 34
6.2. All targeted-set input ratios. M denotes the mirrored construction. Each point is a ratio of twenty-batch medians for one input. 36

A.1. Physical decoding: solid arrows are node links; dashed arrows are header references. Outer chains point up and inward branches down; root slots are empty. Header fingers reference leaves a, d (arrows omitted). Colours and sizes are omitted. 46
A.2. Two-sided split and adoption. The selected component may itself yield an empty side. Sizes come from path spans and header totals; transfer changes header ownership rather than rebinding every leaf. 49

List of Tables

3.1. Increasing and decreasing Step 3 roles in the executable reconstruction. 14

4.1. Selected correctness coverage and its version boundaries. Source identities for the evidence records are listed in chapter B. 25

6.1. Current formal execution. Sample calls are grouped into batches. 33
6.2. Common set: descriptive runtime and paired ratios. Times are milliseconds per call; each input first contributes a median of twenty batch-derived observations. 34
6.3. Common-set ratios grouped by the predetermined nesting classification. These aggregates mix lengths and related constructions. Split counts are per backend in qualification traces. 35
6.4. Targeted set, separately analyzed. Each k group has both lengths and both directions. 35
6.5. All fifteen local diagnostic inputs. T, H and U are the measured, hook-free and additionally unwrapped trees; L is the list. Each row gives ratios of that input's batch medians. Drop is $100(1 - H/T)$, not an additive cost attribution. Original and mirror constructions are related inputs. 37

1. Introduction

1.1. Motivation and Scope

Jordan sorting recovers coordinate order from the traversal order of a simple curve's intersections with a line. The geometric restrictions permit linear-time algorithms [1, 2] when their specialized dynamic-list operations meet the required cost bounds. The problem arises in clipping a simple polygon against a rectangular window: intersections encountered along the polygon boundary must also be ordered along the window's edges. Fung et al. use Jordan sorting as a step in their polygon-clipping algorithm [2].

Replacing these structures with ordinary Python lists can preserve semantics while changing complexity. A structure with a better asymptotic bound may still run more slowly at practical input sizes. This thesis examines the relationship between cost bounds and observed runtime by combining algorithm reconstruction, cost analysis and measurement in an algorithm-engineering study [3, 4].

Since the September progress report, which presented the ordinary-list implementation and its runtime baseline, the scope has expanded to include a finger-tree backend, its amortized analysis and a direct comparison of the two backends. The implementation uses a shared Python core for the 1990 procedure, with ordinary-list and heterogeneous red–black finger-tree backends. With either backend, the core maintains its own sorted output, pair forests and sibling ownership. Inputs are pre-certified valid Jordan sequences with distinct comparable values. Recognition, invalid-input behavior and geometric applications lie outside the study.

1.2. Research Questions

RQ1: Reconstruction and correctness: Can the 1990 procedure be implemented so that it computes the sorted order from its own maintained state, using either ordinary lists or a finger tree, while preserving the invariants of the pair hierarchy?

RQ2: Observed runtime: How do the two backends compare in running time on a fixed set of validated inputs under a paired measurement protocol?

RQ3: Structure and cost: Which properties of the implemented operations yield the linear total-work bound, and how do the concrete operation costs relate to the measured runtime?

The correctness question is addressed through source-linked reasoning and bounded execution checks. The runtime comparison uses paired measurement and independent archive validation. The final question brings these analyses together to relate the implemented operations to the measured runtime.

1.3. Contributions and Findings

The reconstruction specifies the choices of orientation, geometric endpoint and first point. Its tree provides stable identities, endpoint insertion, local split/join and bounded header ownership updates. A manual argument links reversed-pointer representation, locality and two potentials to linear total source-level work. This bound assumes unit-cost comparisons and machine-word operations, expected dictionary lookup and amortized growth.

The current study compares the backends on 140 controlled inputs and a separate twelve-input split set, all of which passed validation on both backends. All 66,560 formal sample calls return correct output in 6,080 batches. The measured tree retains instrumentation hooks, which intercept field reads and writes, with work counting disabled. Its median runtime exceeds the list's on every input. Across the common and targeted sets, the within-input tree/list ratios have medians 2.898 and 3.180, and ranges 1.114–3.535 and 3.147–3.213, respectively.

The common workload has no two-sided splits. The targeted set exercises that path with smaller-side sizes of 3/7/19. These workloads provide complementary views of the implementations: the common set covers endpoint insertion and whole-group transfer, while the targeted set also exercises interior splitting.

1.4. Reading Guide

Chapters 2 to 4 present the background, algorithm reconstruction and implementation. Chapters 5 to 7 describe the experimental method, report the measurements and discuss their limits. Chapter 8 answers the research questions. Chapter A develops the proof, and chapter B indexes its sources and supporting evidence.

2. Background

2.1. Jordan Sequences and the Sorting Problem

Let C be a simple plane curve that crosses the x-axis wherever it meets the axis. Listing the abscissas of the intersection points in the order in which they occur along C gives a sequence

$$z_1, z_2, \dots, z_n. \tag{2.1}$$

Such a realizable sequence is called a *Jordan sequence*. The Jordan sorting problem is to recover the order of the same points along the x-axis, that is, their increasing coordinate order, from their order along the curve [1, p. 170]. The 1990 presentation uses the term Jordan curve for both open and closed simple curves; the property needed here is the absence of self-intersections [2, pp. 85–86].

This formulation gives two distinct orders on one set of points:

Curve order is the input order $z_1, \dots, z_n$ obtained by traversing C.
Axis order is the output order obtained by moving from left to right along the x-axis.

The simplicity of C constrains which permutations of the coordinates are geometrically realizable. The implementation studied in this thesis specializes this model to distinct, mutually comparable coordinate values, as stated in section 4.2.

The problem arose from geometric applications. The 1986 paper relates it to ordering the intersections of a simple polygon with a line and to ordering intersections of two Jordan curves. The 1990 paper develops polygon clipping as its main application and also notes the role of Jordan sorting in simple-polygon triangulation [1, pp. 170–171, 2, pp. 85–86]. In these settings, intersection points can be obtained naturally in boundary or curve order, while the next geometric stage requires their order along another line or boundary.

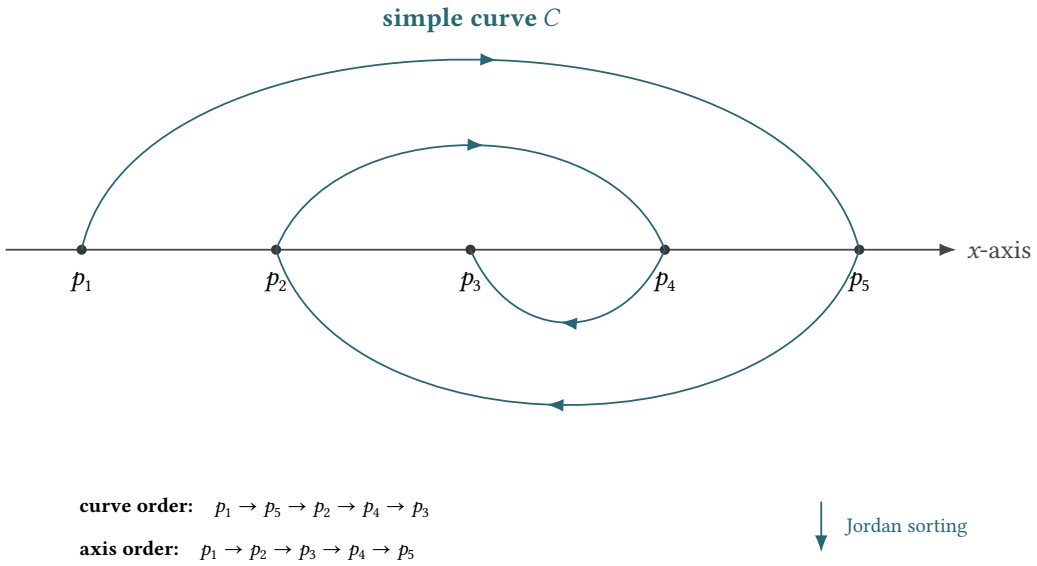


Figure 2.1.: Illustration of the Jordan sorting problem. The simple curve encounters the five intersections in curve order $(p_1, p_5, p_2, p_4, p_3)$, whereas their axis order is $(p_1, p_2, p_3, p_4, p_5)$. Jordan sorting recovers the axis order from the curve order.

2.2. The Structural Restriction Behind Jordan Sorting

Assume, after reflection about the x-axis if necessary, that the first arc lies above the axis. Consecutive intersections delimit alternating arcs above and below it. The pairs

$$\{z_1, z_2\}, \{z_3, z_4\}, \dots \tag{2.2}$$

belong to the upper side, while

$$\{z_2, z_3\}, \{z_4, z_5\}, \dots \tag{2.3}$$

belong to the lower side. Each pair determines the open interval between its two coordinates on the axis.

Two intervals from the same side cannot cross. If one endpoint of one such pair lies strictly inside another pair's interval, then its other endpoint must lie there as well; otherwise the corresponding arcs would intersect. Consequently, same-side intervals are disjoint or nested. Replacing the smaller endpoint by a left parenthesis and the larger by a right parenthesis therefore produces one properly nested parenthesis system for each side [1, pp. 172–173].

The containment relation gives an ordered forest for each parity family. A pair's parent is its smallest same-family enclosing pair, and siblings are ordered by their left endpoints. Adding a dummy pair

spanning $(-\infty, +\infty)$ turns each forest into a rooted ordered tree. These are the upper and lower family trees used by both historical algorithms [1, p. 173, 2, p. 87]. The full notation and executable pair model used in this thesis are developed in section 3.2.

This structure is what makes specialized algorithms possible for valid inputs. While processing points in curve order, the algorithms maintain both a sorted list of the processed coordinates and the two family trees. Inserting the next pair into its family identifies a local position for the new coordinate in the sorted list [1, pp. 173–175]. Efficient sorting therefore depends on maintaining and splitting these ordered family lists.

2.3. The 1986 Linear-Time Algorithm

Hoffmann et al. [1] gave an algorithm that both recognizes and sorts Jordan sequences in $O(n)$ time. It processes the sequence from left to right in curve order, maintaining a doubly linked sorted list and the upper and lower family trees. When a new interval would violate the required nesting, the recognition algorithm aborts; if all points are processed, the maintained list is the sorted result [1, pp. 173–175].

The central data-structural operation is splitting a family list around the new interval. A circular doubly linked representation permits local insertion but can require a scan proportional to the smaller side of a split. Across a sequence of splits this yields an $O(n \log n)$ implementation, no better asymptotically than general sorting [1, pp. 175–176]. The paper obtains its linear bound by replacing those lists with circular level-linked $(2, 4)$ trees.

A level-linked $(2, 4)$ tree stores list items at the leaves of a balanced multiway search tree and connects nodes at equal height. These links support finger searches whose cost depends on the shorter circular distance between the starting item and the target. Together with tree splitting, concatenation, and an amortized analysis, the representation supports the family-list operations within $O(n)$ total time [1, pp. 177–183]. Thus the 1986 result couples two ideas:

1. reduce Jordan recognition and sorting to incremental family-list manipulation; and
2. realize that manipulation with a specialized level-linked data structure.

The second component is essential to the published linear-time bound.

2.4. The 1990 Simplification

Fung et al. [2] retained the linear theoretical bound while simplifying both the primary structure and the operations required of it. A locality lemma constrains re-entry into the region of one parent and its direct children, called a local family in that paper. Its exact grouping rule stores those children in one or two contiguous sibling lists for that parent. As a result, the sorting algorithm needs only three abstract list operations:

1. create a singleton list;
2. insert an item at the front or back of a known list; and
3. split a list at a specified item or key.

In particular, the simplified formulation avoids the 1986 operation that extracts a middle section and concatenates the two remaining ends [2, pp. 86, 88–90].

The algorithm again processes intersections incrementally. It maintains a sorted list, upper and lower family trees, and sibling lists encoding the direct-child order. Neighboring processed coordinates locate the relevant same-family boundaries; insertion of the new pair and any necessary sibling split then determine where the new point enters the sorted list. The exact Step 1/2/3 reconstruction used in this thesis, including the symmetric orientation and endpoint clarifications, is the subject of chapter 3.

For the theoretical implementation, the 1990 paper proposes heterogeneous finger trees. It states constant amortized time for singleton-list creation and boundary insertion, and a logarithmic-in-the-smaller-side amortized bound for splitting. A potential argument charges the split cost across the operation sequence, giving $O(m)$ total time for m list operations and hence an $O(n)$ Jordan-sorting bound [2, pp. 90–91]. The list representation and its operation costs are essential to this bound.

The 1990 paper also states that the sorting method extends to recognition and to the error-correcting Jordan-sorting task used in the polygon-triangulation algorithm of Tarjan and Van Wyk [5], and that neither extension changes its $O(n)$ bound [2, p. 91].

2.5. Related Work and Positioning

2.5.1. Ordered Lists and Finger Search

The specialized list backends used for Jordan sorting belong to a broader line of work on ordered-list representations with fingers and locality-sensitive access. Brown and Tarjan [6] study linear lists with movable fingers, and Brown and Tarjan [7] analyze a 2–3-tree representation for sorted lists

whose costs reflect locality of reference. Huddleston and Mehlhorn [8] develop level-linked weak B-trees for arbitrary sequences of sorted-list updates. The level-linked $(2, 4)$ trees used by the 1986 Jordan-sorting algorithm draw on this line of data-structure research.

These works provide context for the historical linear-time theorem. The ordinary-list backend retains explicit scan, copy, slice and rebinding costs; the heterogeneous backend instead follows the finger-tree approach and is analyzed against its actual source representation in chapter 4.

2.5.2. Jordan Sorting and Geometric Applications

The first symposium publication of the linear-time Jordan-sorting result appeared in 1985 [9]; the 1986 journal article used throughout this thesis gives the expanded algorithm and data-structure account. The later simplification connected Jordan sorting to polygon clipping [2], while the error-correcting task appears in the polygon-triangulation algorithm of Tarjan and Van Wyk [5]. Subsequent work by Bar-Yehuda and Chazelle [10] used a generalized form of Jordan sorting together with point location and interval trees to triangulate collections of disjoint Jordan polygonal chains.

Other work pursued alternatives to the 1986 and 1990 dynamic-list algorithms. Aurenhammer [11] related Jordan sorting to convex-hull computation for certain non-simple polygons. Sojka [12] instead emphasized simpler algorithms intended to be practical at moderate input sizes, including an $O(n \log n)$ worst-case method and an $O(n \log \log n)$ worst-case method. A later average-case study analyzed a new simple algorithm under an explicit random-Jordan-sequence model, identified conditions for linear expected time, and supplemented the analysis with simulation [13]. These alternatives address different trade-offs between structural complexity, worst-case bounds and practical performance. The present study focuses on reconstructing the 1990 procedure.

The geometric applications explain why curve order is a useful input format. The present thesis studies the valid-input sorting step using the existing heterogeneous finger-tree approach.

2.6. Position and Scope of This Thesis

The implementation and evaluation distinguish three tasks:

Geometric validity. A candidate sequence is checked against the Jordan-validity model used for certification before the paper core is timed.

Valid-input sorting. The reconstructed paper control flow maintains its own partial order and recovers the final output from that state.

List representation. Sibling lists use ordinary Python lists or the implemented heterogeneous finger-tree backend, with separate cost analyses.

The empirical study uses oracle-certified valid inputs under the protocol in section 5.1. Certification, sorting and output checking are connected through the interfaces described in section 4.4.

The ordinary-list backend provides a direct representation of the sibling lists, with costs for scanning, slicing, copying and rebinding their contents. The heterogeneous backend requires a source-level account of its physical representation, locality and amortized costs. These two implementations form the comparison developed in the following chapters, with its limits discussed in sections 7.2 and 7.4.

3. Algorithm Reconstruction

3.1. From the Paper to the Implementation

The Jordan sorting problem starts with the intersections of a simple curve and the x-axis in curve order,

$$z_1, z_2, \dots, z_n, \tag{3.1}$$

and asks for the same points in increasing x-coordinate order. The 1990 algorithm of Fung et al. [2] processes this sequence incrementally while maintaining a sorted list and two ordered family trees. Its presentation is compact: the increasing case is described explicitly, the decreasing case is called symmetric, and some endpoint details are left implicit. This chapter states the executable reconstruction used in the thesis.

The reconstruction distinguishes three sources for its rules:

Explicit source rule. A definition or step is stated directly in the 1990 paper, such as the upper/lower family trees, predecessor and successor boundary searches, and increasing Step 3.

Symmetric reconstruction. A decreasing rule is obtained by reflecting the explicit increasing rule: predecessor and successor roles, left and right split sides, child extremes, and insertion direction are exchanged.

Executable clarification. A detail is fixed so that the compact paper description becomes unambiguous code. The geometric child endpoint and the odd-index z_1 output-anchor rule fall into this category.

The executable clarifications are implementation-level interpretations supported by the related 1986 algorithm of Hoffmann et al. [1], reflected counterexamples and bounded exhaustive validation over oracle-certified inputs.

The chapter also separates algorithmic control flow from data-structure complexity. The 1990 paper obtains its linear bound with heterogeneous finger trees. Both thesis backends realize the same high-level operations. Ordinary Python lists incur the quadratic copying lower bound in section 4.6; the finger-tree backend has a separate source-linked representation and amortized-cost argument there.

3.2. Pairs, Intervals, and the Two Families

Input values are assumed to be distinct and mutually comparable. For each $i \geq 2$, define the curve-order pair

$$P_i = \{z_{i-1}, z_i\}. \tag{3.2}$$

The two stored endpoints retain curve order. Their geometric interval is described separately by

$$\ell_i = \min\{z_{i-1}, z_i\}, \qquad r_i = \max\{z_{i-1}, z_i\}. \tag{3.3}$$

A pair P_i encloses a finite point x when

$$\ell_i < x < r_i, \tag{3.4}$$

and it encloses another pair P_j when

$$\ell_i < \ell_j < r_j < r_i. \tag{3.5}$$

After reflecting the curve about the x-axis if necessary, the pairs with even end index belong to the upper family and those with odd end index belong to the lower family:

$$P_2, P_4, P_6, \ldots \in \mathscr{F}^+, \qquad P_3, P_5, P_7, \ldots \in \mathscr{F}^-. \tag{3.6}$$

For pairs in one family, simplicity of the curve implies that their intervals are disjoint or nested. The enclosure relation therefore forms a forest. Each set of siblings is ordered from left to right by increasing ℓ_i.

The reconstruction adds a distinct dummy pair to each family. Each dummy conceptually spans $(-\infty, +\infty)$ and becomes the parent of every pair that has no finite enclosing parent. The two forests are thereby represented as rooted ordered trees, but the upper and lower dummy identities remain separate.

3.2.1. Families and Sibling Lists

Here *parity family* means the entire upper or lower forest $\mathscr{F}^+$ or $\mathscr{F}^-$. For a particular owner p, its *owner region* lies between its arc and its direct children's arcs; a *sibling group* is one stored list of those direct children. The local "family" in the locality argument of Fung et al. [2, pp. 87–88] consists of a parent and its direct children, not the whole parity forest.

The exact grouping invariant G is as follows. Let P_j have the largest end index among the finite members of $\{p\} \cup \text{children}(p)$. If $P_j = p$, all children form one group. Otherwise, the nonempty groups are the children with $r_h \leq z_j$ and those with $\ell_h \geq z_j$, respectively. Empty groups are omitted;

an owner with no children has no group. The dummy is excluded when selecting j. Thus "at most two lists" is necessary but insufficient: the cut must be the endpoint of this particular latest child. The locality and preservation proof is given in section A.2. This locality is what reduces the update to three abstract list operations:

1. create a singleton sibling list;
2. insert a pair at a known list boundary; and
3. split a sibling list at the value of the new point.

The algorithm never requires arbitrary insertion into the middle of a sibling list. A split may produce an empty side, but an empty side is not retained as a live sibling list.

3.3. Maintained State and Prefix Invariant

After processing $z_1, \dots, z_k$, the conceptual state contains:

- a partial-order list S_k containing exactly those k points in increasing value order;
- every finite pair $P_2, \dots, P_k$ in its parity family;
- one parent and one sibling-list owner for every finite pair; and
- the two family-specific dummy roots.

The central prefix invariant is

$$S_k = \mathrm{sort}(z_1, \dots, z_k). \tag{3.7}$$

Here `sort` is mathematical notation for the required order, not an operation invoked by the paper core. The maintained list is bounded by identity-based negative- and positive-infinity sentinels so that predecessor and successor queries are defined at both ends.

The exact grouping invariant G, the family-tree invariant and equation (3.7) are updated together. Pair insertion locates the interval in which z_i belongs, sibling splitting transfers any newly enclosed children, and the resulting family boundary determines the local insertion anchor in S_{i-1}.

3.4. Initialization

Lengths zero, one, and two are handled directly by fixed-size comparison logic. For $n \geq 3$, the incremental state is initialized by ordering z_1, z_2, z_3 with fixed-size comparison logic and creating

$$P_2 = \{z_1, z_2\} \qquad \text{in the upper family,} \tag{3.8}$$

$$P_3 = \{z_2, z_3\} \qquad \text{in the lower family.} \tag{3.9}$$

The upper dummy owns a singleton sibling list containing P_2, and the lower dummy owns one containing P_3. Thus S_3 satisfies equation (3.7), both parity families have a root, and every finite pair has exactly one owner before the main loop begins.

3.5. Boundary Selection

Iteration $i \geq 4$ creates P_i in the family determined by the parity of i. Steps 1 and 2 inspect the immediate neighbors of z_{i-1} in S_{i-1}.

3.5.1. Step 1: Predecessor-Side Boundary

Let v be the predecessor of z_{i-1}. If i is odd and $v = z_1$, replace v by the predecessor of z_1. If the resulting neighbor is the negative sentinel, let A_i be the dummy pair of the current family. Otherwise, let A_i be the already processed same-family pair containing v.

3.5.2. Step 2: Successor-Side Boundary

Let w be the successor of z_{i-1}. If i is odd and $w = z_1$, replace w by the successor of z_1. If the resulting neighbor is the positive sentinel, let B_i be the current family dummy. Otherwise, let B_i be the processed same-family pair containing w.

A processed point can belong to both an upper and a lower pair. The parity of i selects the relevant incident pair without using a global rank map. The special treatment of z_1 is necessary because z_1 belongs to P_2 but to no lower-family pair. These two neighbor adjustments select family-tree boundaries; they are distinct from the output-anchor adjustment introduced in section 3.8.2.

3.6. Step 3: Increasing Orientation

Assume first that $z_{i-1} < z_i$. Let $N = P_i$ denote the new pair. The predecessor-side pair A_i controls where N enters the family tree, while the successor-side pair B_i controls whether an existing sibling list must be split.

3.6.1. Step 3(a): Insert the New Pair

If A_i encloses z_{i-1}, then N becomes its child in a new singleton sibling list. Otherwise, A_i is the last item of its sibling list; N is inserted immediately after A_i and inherits the same parent. In either case, N obtains exactly one parent and one sibling-list owner.

3.6.2. Step 3(b): Split Off Newly Enclosed Children

If B_i encloses z_{i-1}, N acquires no child list. Otherwise, B_i is the first item of an existing sibling list L. Splitting L at z_i produces a left side and a right side. For a valid input, no pair may straddle the split value: both endpoints of each child lie on one side. The left side, whose pairs are newly enclosed by N, becomes a child list of N; the right side remains with the previous owner of L.

3.6.3. Step 3(c): Insert the New Point

If N acquired no children, the base output anchor is z_{i-1}. Otherwise, let C be the rightmost child pair of N and choose the geometric right endpoint $r(C)$ as the base anchor. After the possible odd-index adjustment in section 3.8.2, insert z_i immediately after the output anchor.

3.7. Step 3: Decreasing Orientation

For $z_i < z_{i-1}$, the reconstruction uses the order reflection of the increasing case. This is the executable meaning assigned to the 1990 paper's statement that the other case is symmetric.

Step 3(a) now uses B_i. If B_i encloses z_{i-1}, N becomes its child in a singleton sibling list. Otherwise, B_i is the first item of its list; N is inserted immediately before it and inherits its parent.

Step 3(b) uses A_i. If A_i encloses z_{i-1}, no split is needed. Otherwise, A_i is the last item of its sibling list. The list is split at z_i; the right side is transferred to N, while the left side remains with the old owner.

For Step 3(c), the no-child base anchor is again z_{i-1}. With children, let C be the leftmost child pair of N and use its geometric left endpoint $\ell(C)$. After the reflected odd-index adjustment, insert z_i immediately before the output anchor.

The complete reflection is summarized in table 3.1.

Table 3.1.: Increasing and decreasing Step 3 roles in the executable reconstruction.

Operation	Increasing	Decreasing
Step 3(a) boundary	predecessor-side A_i	successor-side B_i
Boundary insertion	after last sibling	before first sibling
Step 3(b) boundary	successor-side B_i	predecessor-side A_i
Acquired split side	left	right
Child extreme	rightmost child	leftmost child
Geometric anchor	right endpoint	left endpoint
Output insertion	after anchor	before anchor

3.8. Executable Anchor Clarifications

The point insertion in Step 3(c) is where the compact paper notation requires the most care. Two independent distinctions are necessary.

3.8.1. Curve Endpoint Versus Geometric Endpoint

The 1990 increasing step refers to the rightmost child $\{z_{m-1}, z_m\}$ and inserts the new point after z_m. A pair record, however, stores its endpoints in curve order, and its second curve endpoint is not necessarily its geometric right endpoint. Reading z_m mechanically as the second stored endpoint fails on valid reflected orientations.

For example, consider

$$(3, 2, 1, 4). \tag{3.10}$$

At $i = 4$, the acquired child is $P_2 = \{3, 2\}$. Inserting 4 after the second curve endpoint 2 would yield $(1, 2, 4, 3)$. Inserting after the child's geometric right endpoint 3 yields the correct order. The decreasing counterpart

$$(2, 3, 4, 1) \tag{3.11}$$

similarly requires insertion before the child's geometric left endpoint 2, not before its second curve endpoint 3.

The reconstruction therefore uses $r(C)$ for the increasing branch and $\ell(C)$ for the decreasing branch. This choice agrees with the earlier family-list account of Hoffmann et al. [1], in which both endpoints occur in sorted family lists and the new point is inserted next to the located boundary item. This is the geometric-endpoint clarification used in the implementation.

3.8.2. Odd-Index z_1 Output Anchor

Steps 1 and 2 compensate for the fact that z_1 has no lower-family pair. That boundary-pair correction does not cover every final output insertion. The 1986 description records a separate anomaly: on an odd iteration, if z_1 lies between the ordinary family-list boundary and the new point, the new point is inserted next to z_1 [1, p. 175].

The executable increasing rule is

$$\text{if } i \text{ is odd and } a_i < z_1 < z_i, \quad a_i \leftarrow z_1, \tag{3.12}$$

where a_i is the geometric base anchor. The decreasing reflection is

$$\text{if } i \text{ is odd and } z_i < z_1 < a_i, \quad a_i \leftarrow z_1. \tag{3.13}$$

For the decreasing sequence prefix

$$(1, 2, 3, 4, 6, 7, 0), \tag{3.14}$$

the ordinary geometric base anchor at $i = 7$ is 2. Inserting 0 before 2 would leave 1 on the wrong side. Because $0 < z_1 = 1 < 2$, the output anchor is changed to z_1, and insertion before it gives the correct prefix order. The reflected sequence $(6, 5, 4, 3, 1, 0, 7)$ exercises equation (3.12). These cases demonstrate why boundary selection and output-anchor correction must be modeled as separate stages.

3.9. Ownership Transfer and Failure Semantics

A sibling split changes several linked facts at once: the old list ceases to exist, up to two nonempty output lists are created, the retained side stays with the old parent, and the acquired side moves under N. Each moved pair must belong logically to exactly one output sibling group and have that group's owner as its parent. The ordinary backend updates per-pair ownership fields, whereas the tree backend updates headers without rebinding every leaf. Conservation and unique ownership are required after a successful transition.

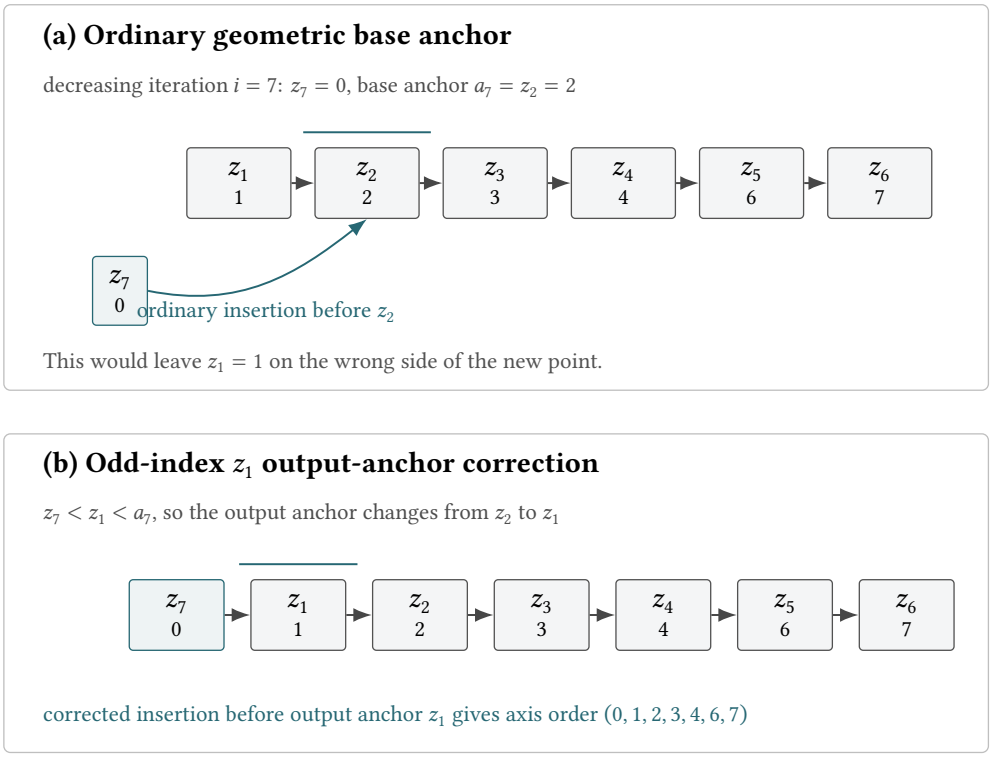


Figure 3.1.: Local output-anchor selection in decreasing Step 3(c) for the prefix $(1, 2, 3, 4, 6, 7, 0)$. The ordinary geometric base anchor is z_2. Because the odd-index condition $z_7 < z_1 < z_2$ holds, the output anchor is corrected to z_1 before z_7 is inserted.

The ordinary backend first forms a split plan without changing live state; its commit checks the plan, parent and partition before updating ownership. The tree instead uses a planned transfer source, path-local splitting and header adoption, without materializing that list-based plan. Both must restore the logical ownership conditions before completing the iteration. In the revised implementation, an update or postcondition error terminates sorting. Updates already performed are not rolled back, and the failed state must be discarded. No partially updated state is accepted as a completed iteration or returned as a sorting result.

The earlier implementation used rollback on these error paths. The version history is described in section 4.1, and the concrete update operations in section 4.3.3.

3.10. End-to-End Procedure

The full valid-input sorting procedure is:

1. Materialize the input once and handle lengths zero, one, and two directly.
2. For $n \geq 3$, initialize S_3 by fixed-size ordering of the first three points, then create P_2, P_3, the two dummies, and their singleton sibling lists.
3. For each $i = 4, \ldots, n$:
   Step 1. Compute predecessor-side boundary A_i.
   Step 2. Compute successor-side boundary B_i.
   Orient. Choose increasing or decreasing Step 3 from the comparison of z_{i-1} and z_i.
   Update. Insert P_i, split and transfer any enclosed children, and insert z_i using the resulting geometric output anchor.
   Commit. Advance the processed prefix only after the output insertion succeeds.
4. Return the values obtained by traversing S_n.

At the end of each successful iteration, the required online conditions are:

- S_i contains each processed point exactly once and is strictly increasing;
- every processed finite pair exists in the family dictated by its parity;
- every finite pair is a member of exactly one live sibling group, whose owner is its parent;
- sibling order follows geometric left endpoints, parent chains are acyclic, and each chain reaches the appropriate dummy root; and
- split outputs are disjoint and preserve the input pair set.

The final return value is recovered from the maintained order S_n. The certification wrapper that establishes the input precondition is described in section 4.4.

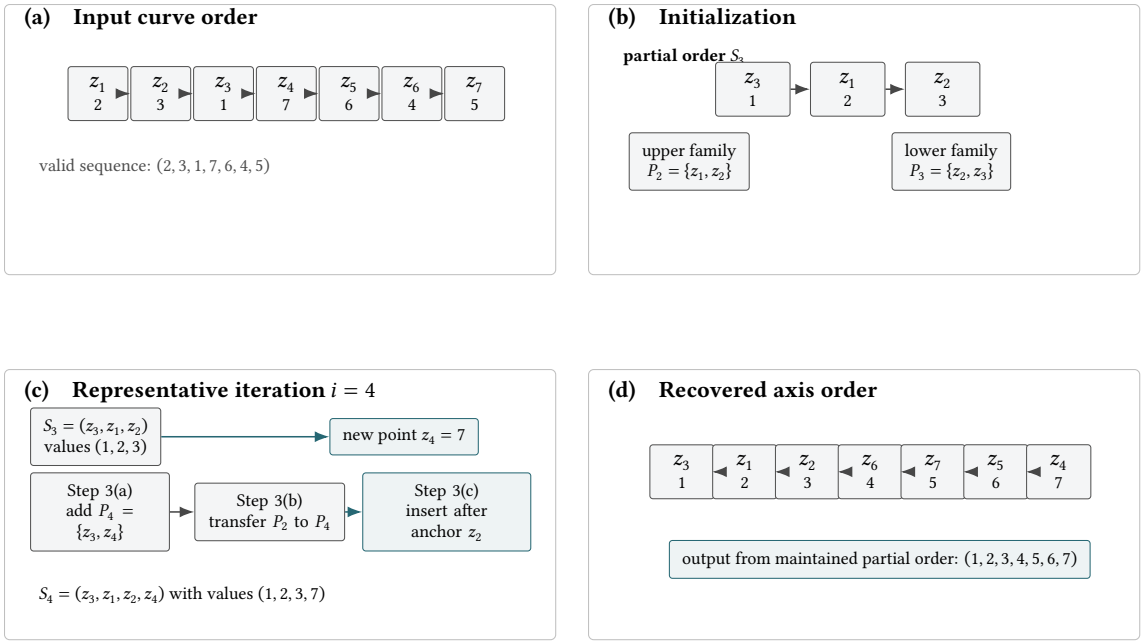


Figure 3.2.: Running example on the valid sequence $(2, 3, 1, 7, 6, 4, 5)$. The panels show the input, initialization, the representative update at $i = 4$, and the recovered axis order. At $i = 4$, P_4 acquires P_2 in Step 3(b), then Step 3(c) inserts z_4 after the geometric anchor z_2.

3.11. Correctness and Operation Costs

The source-level correctness argument rests on the nesting property of same-family pairs and the locality lemma: the boundary pairs found beside z_{i-1} identify where P_i belongs, and newly enclosed children form a contiguous split side. The executable reconstruction turns those facts into the prefix and ownership invariants above. Focused counterexamples fix the ambiguous anchors; mirrored tests check the decreasing reconstruction; and full state diagnostics compare the maintained order and family forest with a deterministic replay.

The cost of these steps depends on the backend. Hoffmann et al. [1] use level-linked search trees, while Fung et al. [2] reduce the required operations to singleton-list creation, boundary insertion, and splitting, then implement them with heterogeneous finger trees. The ordinary backend preserves those abstract operations using Python lists. A sibling split scans and materializes list contents, both insertion boundaries copy the current list, and ownership rebinding can depend on list size. Membership and ancestor checks can also scan lists during sorting. The tree backend instead uses the physical operations described in chapter 4, where the costs of both representations are analyzed.

4. Implementation

4.1. Architecture and Versions

The code separates an oracle-backed reference pipeline from the shared-core reconstruction of Fung et al. [2]. The reference function `simplified_jordan_sort` certifies the input, constructs family trees, statistics and a trace, and returns the oracle's sorted output. In contrast, `paper_jordan_sort_valid` executes the reconstructed algorithm and returns `state.partial_order.to_list()`. Its core does not use the oracle's output or a global rank map. It assumes a pre-certified valid Jordan sequence with distinct, mutually comparable values.

Both current backends propagate errors without rollback. The paired timings in chapter 6 measure these programs; the earlier rollback-enabled study is documented in its historical section. Source bindings are listed in chapter B.

Chapter 3 specifies initialization, both orientations of Step 1/2/3, and executable anchor clarifications. Here the focus is state representation and maintenance. Both backends share the sorting loop, while their representations determine how they perform each list operation.

4.2. Data Representation

An immutable `PointRef(paper_index, value)` separates one-based paper identity from a point's comparable value. A `PairRecord` retains curve endpoint IDs and family membership; geometric endpoints are obtained by comparing their values. The distinction and its consequences for output anchors are specified in section 3.8.1.

`PaperJordanState` holds these records, the processed-prefix length, the partial order, the end-index-to-pair mapping, the sibling backend and two family roots. It also holds a fixed execution policy, optional trace and counters, and mandatory stage results. Sorting and diagnostics use the same runner and mirrored orientation branches.

4.2.1. Partial Sorted Order

`SortedOrderList` is doubly linked, with identity-based infinity sentinels and a point-ID-to-node mapping. Neighbor access and insertion use local links. Before mutation, insertion checks that the value lies strictly between its adjacent values. Final traversal excludes the sentinels and returns the original values. Mapping access uses Python dictionaries and is charged at its expected cost.

4.2.2. Sibling Lists and Ownership

Each `SiblingList` stores ordered finite pair IDs and one owner. A pair caches its parent and containing-list IDs, together with its ordered child-list IDs. Every owned finite pair occurs in exactly one live list, whose owner equals its parent. A parent has at most two ordered child lists; finite parent chains must reach the same-family dummy without cycles. The family dummies are roots, not finite list items.

The ordinary backend retains this eager representation. The shared core uses boundary-list, owner and extreme-child access interfaces instead of reading those fields directly. General pair-liveness checks do not require the pair to be a list endpoint.

The tree backend stores stable pair handles in external leaves, with internal separator nodes, two endpoint fingers and a list header. Outer rib pointers point toward the root; interior pointers point toward children. Separators store the maximum left key. Rank is black height excluding external black leaves. For internal nodes, span counts all leaves in the INTERIOR role, the inward branch on a rib, and zero in the ROOT role. External leaves always have span one, including a singleton root. The header separately stores total size. Ownership belongs to the header and at most two parent child slots, avoiding per-leaf rebinding. Boundary slots only identify designated endpoints; they are not arbitrary-owner lookup caches.

4.3. Core–Backend Interface and Update Semantics

4.3.1. Initialization, Boundaries and Stage State

The initialization in section 3.4 materializes all point records but orders only the first three values; shorter inputs use fixed-size comparisons. Subsequent iterations use the predecessor and successor boundaries defined in section 3.5. The backend handles singleton creation, boundary insertion, splitting and ownership transfer; the partial-order structure handles point insertion.

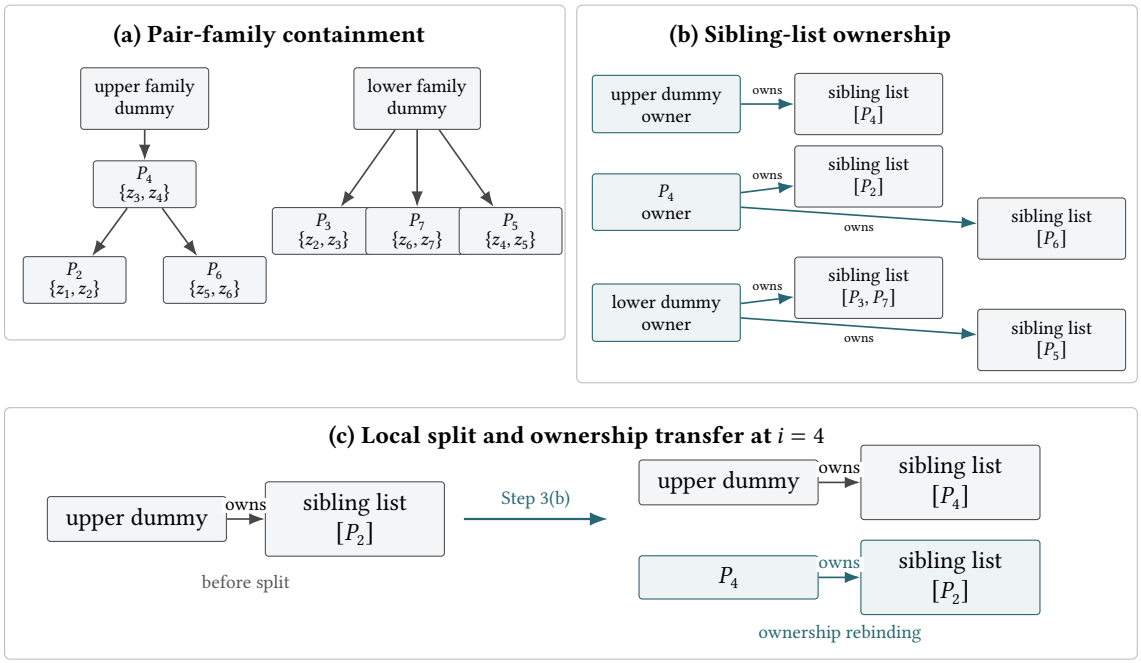


Figure 4.1.: Pair families and sibling-list ownership in the example of figure 3.2. Step 3(b) transfers P_2 from a list owned by the upper dummy to a child list of P_4.

Boundary selections and Step 3 results are stored even with trace disabled. Later stages consult them to reject incorrect order, repeated stages and inconsistent parameters. They are execution state, not merely optional logging. Optional backend-work observations are stored separately from these mandatory results; complete logical snapshots belong to the audit path.

4.3.2. Pair Insertion and Failure Propagation

For the insertion rules in sections 3.6.1 and 3.7, the core checks the boundary, registers the new pair and calls singleton creation or boundary insertion. A later error may leave that registration in place. The exception propagates; the state is discarded rather than restored or reused. This contract applies to both orientations.

4.3.3. Split Plans and Ownership Transfer

The ordinary backend implements the partition specified in sections 3.6.2 and 3.7 using an immutable `SplitPlan`. It records the original sequence and both ordered partitions before mutation. The plan sup-

ports stale-plan rejection, order checks and pair-set conservation; these functions remain necessary without rollback.

Commit checks the plan and ownership preconditions, prepares nonempty output lists and affected child slots, retires the input list, publishes the new lists, and rebinds the pairs. Touched-state postconditions are checked before success is returned. Rejection covers stale plans, straddling pairs, ownership mismatches, cycles, unowned parents and a third child list. Errors during mutation or postcondition checks propagate without restoring earlier state.

The tree starts boundary insertion at a finger and repairs locally. A split with both sides nonempty races from both ends, detaches a component, splits its downward representation and joins by black height. Path sizes and header totals supply output sizes without enumerating leaves. Every successful split, including a one-sided result, retires its source ID and allocates fresh IDs for nonempty outputs; boundary insertion preserves its live ID. Pair handles, leaves and existing descendant groups survive transfer. Before Step 3(a), the core identifies the other group's transfer source without splitting it. Step 3(b) consumes that exact source; pending split/transfer state must clear before Step 3(c) inserts the output point.

4.3.4. Point Insertion and Completion

The backend exposes the relevant extreme child; the core selects its geometric endpoint and applies the separate odd-index z_1 anchor rule of section 3.8.2. The clarifications and counterexamples are given in section 3.8. The partial order performs the resulting adjacent insertion. Only after it succeeds does Step 3(c) advance `processed_count`.

4.4. Certification, Diagnostics and Replay

The wrapper `certified_paper_jordan_sort` runs the oracle and rejects invalid candidates before entering the pure sorter. The *minimal core* is the sorting call with optional full-state checks, operation counters and tracing disabled; necessary local checks remain active. Certification is outside this call's timing scope. An internal exception is propagated by both sorting and diagnostic entry points: neither retries, returns a partial output, nor falls back to the oracle or reference pipeline. Output recovery is reached only after the runner succeeds.

`paper_jordan_diagnostics_valid` selects `checked` and audits initialization and every completed prefix. It checks point identity and values, partial-order links and order, exact pair membership, ownership and parent chains, stage records, trace and metrics. *Same-backend replay* reconstructs a fresh state

using the same core, backend type and immutable configuration. It compares backend identity, partial order, registries, stage results, trace, work observations, metrics, and logical and physical snapshots. Unavailable backends fail explicitly rather than reverting to ordinary lists. Replay checks same-core state consistency; external sorting checks the final output.

Qualification, the pre-timing validation described in section 5.2, advances two independently initialized checked instances through the same pausable core, audits and compares every completed prefix, then performs one final fresh checked replay. A separate fresh minimal call checks the measured configuration. Snapshots reuse one audit within a validation call, never a cross-call "already checked" cache. Old stage and trace records remain checked.

Failure-injection tests separately check propagation from pair insertion and split postconditions, including checks retained in `minimal`. Public-call tests require one split attempt and no output recovery after failure.

4.4.1. Correctness Coverage

Table 4.1 separates operation tests, complete sorting runs and measured workloads. Each development result refers to its recorded source snapshot; the qualification rows identify the measured versions.

Table 4.1.: Selected correctness coverage and its version boundaries. Source identities for the evidence records are listed in chapter B.

Level	Covered behaviour and limit	Evidence record
Tree operations	53 test methods: insertion, split/adopt, both outputs updated and split again, existing descendants and stable identities. Near-nephew deletion repair has operation-level coverage; complete-sort coverage is not established.	Operation tests
Complete sorting	18 fixed balanced, skew and repeated constructions, $n = 128/129, 256/257, 512/513$; checked/minimal and three backend configurations. Correctness and resource evidence only.	Structural campaign
Common group	140 inputs qualified, including fresh minimal calls; 3,788 splits per backend, all with an empty side. Formal timing therefore does not exercise interior splitting.	Common-set qualification
Targeted group	12 inputs qualified separately; 12 two-sided splits per backend, source size 61, smaller side 3/7/19. Later direct insertions reach only the retained side.	Split-set qualification
Seeded supplement	40 fixed inputs up to $n = 1500$: list/tree minimal and tree trace checked against sorted output; eight small cases also checked with fresh-prefix replay. 678 two-sided splits; no timing evidence.	Seeded supplement; October 20 protocol

The 18-input campaign records 460 splits, 230 two-sided splits, 666 later insertions into earlier split outputs and 212 re-splits, per checked configuration's original runs. Within it, `repeated-512` contributes respectively 126, 63, 189 and 62 operations; its two-sided splits always have smaller side one. The 512-point balanced and skew cases split 253 siblings into 126 + 127 and 1 + 252; the 513-point cases mirror these constructions.

The seeded generator extends from z_{i-1} into an axis gap such that no same-parity arc has exactly one endpoint between z_{i-1} and the new point; final axis-order ranks become the input values. With probability b it chooses uniformly among legal gaps; otherwise it follows a direction (reversed with probability 0.08), choosing its nearest or farthest legal gap with equal probability, or all legal gaps if that side is empty. Seeds 11–14 each generate sizes 128/512/513/1500 in that order, at $b = 0.05, 0.25$ for each size; seeds 21–22 each generate 32/33/64/70 at $b = 0.1$. Each seed uses one sequential Python `random.Random` stream; all forty inputs are saved and independently certified by the oracle. Thirty-eight contain a two-sided split, with maximum smaller side 13. The eight smaller inputs also receive checked validation. This supplement extends the tested operation histories; the common and targeted groups supply the formal timing data. Archive locators and the source-hash distinction are in section B.1.

4.5. Execution Policies and Observation Costs

Five immutable policies control trace recording, selected operation counters and complete backend validation after split commits. `checked` enables all three; `minimal` disables them. Intermediate policies isolate observation costs. All use the same Step 1/2/3 sequence and retain stage results, necessary local checks, input materialization and output recovery. The revised implementation has no rollback under any policy.

The tree accesses node, handle and header fields through *instrumentation hooks*: methods that intercept field reads and writes and record work when the detailed work ledger is enabled. This ledger has a separate configuration switch; minimal mode alone does not disable it. Qualification, calibration and formal measurement explicitly select `detailed_work=False`, suppressing work-count accumulation, ledger snapshots and per-operation reports. The hooks remain in place, so field access still incurs Python method dispatch and field-membership and ledger-switch checks. In particular, `_Tracked.__getattribute__` and `__setattr__` still intercept accesses before testing the ledger switch; local geometry checks, pending-state checks, and operation and phase wrappers also remain. Throughout the empirical chapters, "measured tree" means this configuration with retained instrumentation hooks.

The archived studies instead timed the rollback-enabled implementation, including recovery-state preparation in `minimal`; a successful call need not actually roll back to incur that preparation. The historical scope is specified in section 5.8.

For each successful split, the ordinary backend reports an increment equal to the input-list length for both `split_items_scanned` and `split_items_copied`. These logical split-volume measures omit boundary-insertion copies, membership scans and repeated validation/materialization passes. The shared core accumulates each backend's reported work under an identified metric schema. This keeps logical volumes separate from observed field accesses and elapsed time.

4.6. Operation Costs and Complexity

In the ordinary backend, either boundary insertion constructs a new Python list containing the existing IDs and the new pair. Rear insertion is not an in-place `list.append()`. Both orientations copy work proportional to the current sibling-list length. Split additionally scans and partitions the list, materializes its outputs and rebinds pair ownership.

"Local" describes the objects a safety check concerns, not constant cost. The ordinary backend's liveness check uses `pair_ids.count(pair_id)` for membership, and backend parent checks scan the containing list at each visited ancestor. For an ancestor chain A, this contributes $O(\sum_{a \in A} |L(a)|)$, where $L(a)$ contains a, rather than a constant per ancestor. These costs remain in `minimal`; complete checked validation and prefix replay add separate untimed work.

The valid increasing input $(1, 2, \ldots, 2m)$, $m \geq 2$, exposes a quadratic copying lower bound even without split. Starting from one pair per family, boundary insertions grow the sibling lists to m and $m - 1$ while Step 3(b) skips splitting. The number of existing pair IDs copied is

$$\sum_{j=1}^{m-1} j + \sum_{j=1}^{m-2} j = (m-1)^2. \tag{4.1}$$

Thus this input family incurs $\Omega(n^2)$ copying, including in `minimal`. This lower bound accounts for insertion copying alone.

For the tree backend, the source-level proof in chapter A accounts for a different set of operations: decoding and local repair preserve the representation, exact grouping supplies legal endpoint calls, and one potential pays propagation across mixed insertion/split histories. A second, list-size potential pays the smaller-side split terms over the sorting run. Theorem A.10 includes the shared core, ownership and output work under its stated computation model.

5. Experimental Method

5.1. Study Design and Versions

The current ordinary-list and tree implementations have completed qualification, separate batch calibration and formal paired measurement. Current results in chapter 6 use the measured version indexed in chapter B; the older rollback-enabled ordinary-list study is described in the historical section of this chapter.

5.2. Current Inputs and Qualification

Before any timing, every input was run on both backends with all state audits enabled; we call this step *qualification*. Only inputs that passed qualification on both backends were measured.

The common set was fixed in advance and contains 140 unique rank inputs at ten lengths: 32, 33, 64, 65, 128, 129, 256, 257 and 512, 513, with two strict-flat and four each of low, medium and heavy nesting per length. Fixed constructors, seeds, candidate budgets, rank deduplication and first-eligible retention preceded timing. All 140 cases pass both backends' qualification checks. Their 3,788 recorded splits per backend all have an empty output side. The common set therefore exercises endpoint insertion and whole-group transfer.

A separate targeted set uses the existing construction at $n = 128, 129$, $k = 3, 7, 19$, with each direction and its mirror: twelve cases, all qualified. Actual traces contain twelve two-sided splits per backend, with smaller sides 3, 7 and 19 occurring four times each. Each source has size 61 and a finite owner; later direct updates affect only the retained side. The 152 inputs are reported as two datasets.

For each input/backend, qualification advances two independent checked instances, compares every completed prefix, and executes a third fresh checked replay at the end. Another fresh minimal call uses the intended timing configuration. Each output is compared to external sorting, without feeding that expectation into the core. This protocol is recorded as version 2 in the experiment archives. Compared with the earlier protocol, which reconstructed a fresh state at every prefix, it retains prefix comparisons but reduces repeated initialization.

5.3. Compared Implementations and Call Scopes

The current comparison uses the same public Jordan core with ordinary-list or heterogeneous-finger storage, both minimal; the tree explicitly sets `detailed_work=False`. Every call initializes fresh state and recovers its output. Production hashes match the qualified versions; changes require renewed validation of the affected evidence.

The retained instrumentation hooks described in section 4.5 are inside the measured interval. The experiment compares these two concrete programs, including their different checks and object-access machinery.

5.4. Timing, Calibration and Scheduling

Current timing uses `time.perf_counter_ns()`. The interval includes fresh initialization, the main loop, output recovery, the batch loop and output append. External comparisons, pre-batch garbage collection, compression and persistence are outside it. GC remains enabled, with thresholds recorded; necessary local checks remain enabled.

Calibration tries $B = 1, 2, 4, 8, 16, 32, 64, 128$ in order, three batches per backend at each attempted value. It chooses the first common B for which both median batch durations reach 10 ms, irrespective of their ratio. All attempts and outputs are retained. With the calibration version, the common set selected B=1,2,4,8,16,32 for respectively 24,20,20,28,21,27 inputs; all twelve targeted inputs selected B=8. Independent reconstruction accepted both campaigns, within their separate 600-second budgets.

5.4.1. Scheduling and Noise

Calibration alternates the first backend by case index, candidate index and repeat. The executed formal protocol shuffles cases with seed `20261015`, uses five warm-up calls per backend and twenty paired batches at each predetermined B, alternating first position by case index and round. Each backend is first ten times. Failures stop subsequent cases; timeouts preserve committed batches and leave explicit unrun statuses. No outlier removal or speed-based input replacement is allowed.

Alternating the first backend reduces fixed-position bias, while thermal and operating-system effects remain possible sources of noise [14]. Repetition alone does not establish adequate statistical power [15].

5.4.2. Recorded Environment

Each current archive binds source hashes, Python executable/version/flags, platform, CPU, RAM, power observations, GC thresholds and monotonic-clock metadata. Calibration and formal execution must match source and recorded machine/runtime settings. Tests, qualification and compilation were not run concurrently with calibration or formal sampling. The current formal archives record Apple M4/arm64, 16 GB RAM, ten logical CPUs, macOS 26.6 and CPython 3.12.14; their captured power state is battery at 100%.

5.5. Aggregation and Statistical Units

The batch is the statistical unit. Dividing its duration by B gives a per-call duration for that batch. The protocol reports twenty such observations per input/backend, their median, R7-interpolated quartiles and IQR, and the paired ratio of backend medians. The inputs are controlled structural constructions; mirror pairs and related constructions are dependent. Cross-input summaries therefore describe these fixed datasets.

5.6. Experimental Records and Validation

Execution identities, source bindings and archive locations are collected in chapter B. Formal execution uses the qualified production code, fixed batch-size tables and required matching environment fields. Each archive retains the identity of the execution that produced it.

Records are individually compressed and atomically committed, so termination preserves earlier completed batches. A manifest binds configuration, records and environment. Campaign wall time includes untimed checking and storage.

5.6.1. Fail-Closed Validation

The current archive checker verifies schemas, source blobs, input identities, outputs, event order, sizes, owners, counters and list-ID lifetimes. A live-ID map checks current membership; a separate seen-ID set prevents a singleton or split output from reusing any retired ID. Boundary insertion keeps its live ID. Read-only rechecks accept both existing datasets without rerunning sorting. The measurement validator independently reconstructs candidate order, every attempt, the first qualifying common B and output correctness. For formal archives it reconstructs warm-ups, paired sample

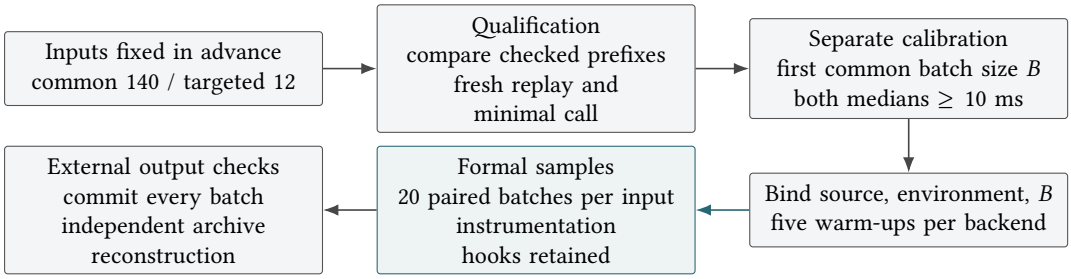


Figure 5.1.: Current evidence flow. Qualification, calibration and formal sampling have separate identities. The same predetermined inputs and per-input B reach the samples; independent reconstruction checks each archive.

order, twenty-batch medians, R7 quartiles and within-input ratios. Acceptance requires all of these checks to pass.

5.7. Study Stages

Qualification checks the candidate implementations, calibration fixes the batch sizes, and formal measurement collects the runtime samples. The formal runs followed the manual proof review and passed archive validation. Their samples supply the current performance tables.

5.8. Historical Study: Separate Scope

The earlier study used 60 inputs: flat, nested and ten seeded incremental cases at each of five even lengths 32–512. It compared Python sort, the oracle-backed reference pipeline and the rollback-enabled ordinary-list core, with five warm-ups and twenty individual calls per case/algorithm. The latter core paid recovery preparation even on successful calls; the reference also included recognition and structural analysis. Its GC-disabled timed calls, CPython 3.12.4, median-of-halves quartiles and separate scheduling differ from the current protocol. The 3,600 historical rows and full method remain bound to their original archive (section B.1); they are not pooled with the current comparison or used to infer a rollback-removal benefit.

6. Results

6.1. Execution Summary and Output Checks

The current paired experiments used the measured version, whose Python sources are identical to those of the calibration version. The common and targeted datasets remain separate. Both archives passed independent reconstruction of inputs, configuration, schedule, outputs, predetermined batch sizes and statistics. An additional analysis rebuilt the per-input medians and quartiles from every original sample record and agreed with the validator's values. The manifest-bound records and reproducible analysis are indexed in section B.1.

Table 6.1.: Current formal execution. Sample calls are grouped into batches.

Dataset	Inputs	Sample batches	Sample calls	Warm-up calls
Common	140	5600	62720	1400
Targeted split	12	480	3840	120

Each input/backend contributes twenty sample batches after five warm-up calls. All 66,560 sample calls and 1,520 warm-up calls returned the expected output; no case was erroneous, timed out, skipped or replaced. These output checks followed qualification and were performed outside the timed intervals.

6.2. Common-Set Runtime and Paired Ratios

The runs use Apple M4/arm64, 16 GB RAM, ten logical CPUs, macOS 26.6 and CPython 3.12.14, with GC enabled at thresholds 700/10/10. The captured power state is battery power at 100%. Both backends use the same public minimal sorting scope, and the tree disables work counting while retaining instrumentation hooks (section 4.5). Initialization, sorting, output recovery, the batch loop and output append are included; expected-output comparisons and persistence are excluded.

Table 6.2 gives medians across input-specific medians. The last column is the median of ratios formed within each input, rather than the ratio of the aggregate time columns. Ratios above one mean a slower tree implementation under this protocol.

Table 6.2.: Common set: descriptive runtime and paired ratios. Times are milliseconds per call; each input first contributes a median of twenty batch-derived observations.

n	Inputs	List (ms)	Tree (ms)	Median ratio
32	14	0.612	1.934	3.153
33	14	0.620	1.957	3.086
64	14	1.291	3.646	3.030
65	14	1.275	4.012	3.089
128	14	2.642	8.082	2.898
129	14	2.772	8.002	2.890
256	14	6.146	13.999	2.625
257	14	6.218	15.427	2.600
512	14	16.390	29.041	2.098
513	14	16.136	33.791	2.123

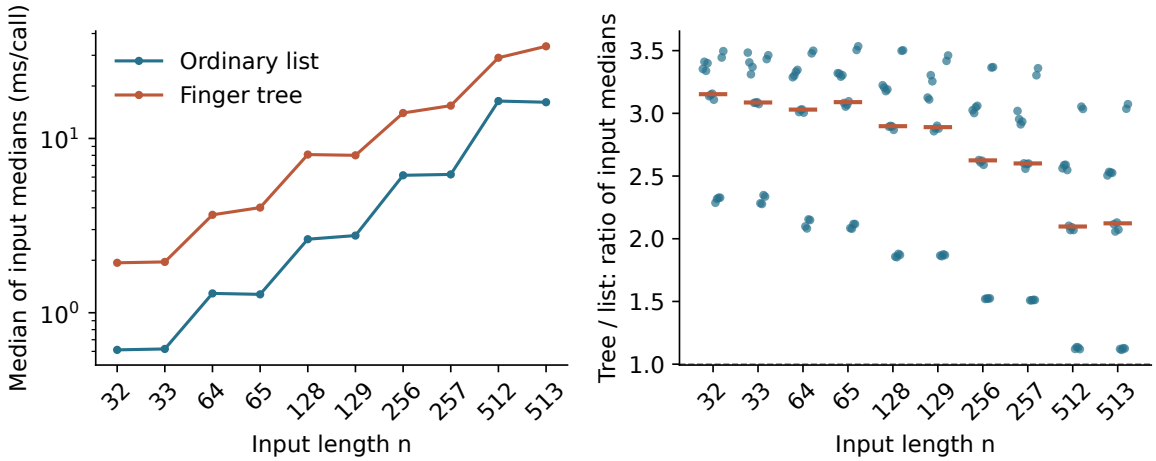


Figure 6.1.: Current common set. Left: medians of input medians, with a logarithmic time axis. Right: all fourteen input ratios per length; horizontal marks are their medians. Horizontal offsets separate points. The dashed line denotes equal runtime.

Across the 140 inputs, tree/list ratios range from 1.114 to 3.535, with median 2.898 and cross-input quartiles 2.117 and 3.210. Every input has a tree median above its paired list median. The size-stratum median decreases from 3.153 at $n = 32$ to 2.123 at $n = 513$, with local nonmonotonic changes.

Table 6.3.: Common-set ratios grouped by the predetermined nesting classification. These aggregates mix lengths and related constructions. Split counts are per backend in qualification traces.

Stratum	Inputs	Median ratio	Range	Two-sided splits
Strict-flat	20	3.439	3.035–3.535	0
Low nesting	40	3.198	2.505–3.484	0
Medium nesting	40	2.886	2.058–3.157	0
Heavy nesting	40	1.868	1.114–2.348	0

6.2.1. Structural Strata

The heavy group has the smallest descriptive ratios. Its constructors differ from the other groups in several features, so the ratios cannot be attributed to nesting alone. The qualification traces contain 3,788 split calls per backend, all with an empty-side outcome. The nesting classes thus compare workloads involving insertion, whole-group movement and ownership updates; interior splitting is examined in the targeted set below.

6.3. Targeted Split Set

The twelve targeted inputs at $n = 128, 129$ contain the intended nonempty splits: the smaller side is 3, 7 or 19, each represented by four inputs. Source size is 61 under a finite owner. The traces also cover subsequent retained-side insertion, but not updates to both outputs in every case. These operation histories come from the qualification traces.

Table 6.4.: Targeted set, separately analyzed. Each k group has both lengths and both directions.

k	Inputs	List (ms)	Tree (ms)	Median ratio
3	4	2.654	8.432	3.177
7	4	2.673	8.478	3.163
19	4	2.560	8.171	3.207

Ratios range from 3.147 to 3.213; the median is 3.180, with cross-input quartiles 3.165 and 3.189. The tree is slower on all twelve cases, and the ratios show no monotonic relation with k. These measurements cover complete sorting calls, including insertion, ownership updates and output recovery alongside the targeted split.

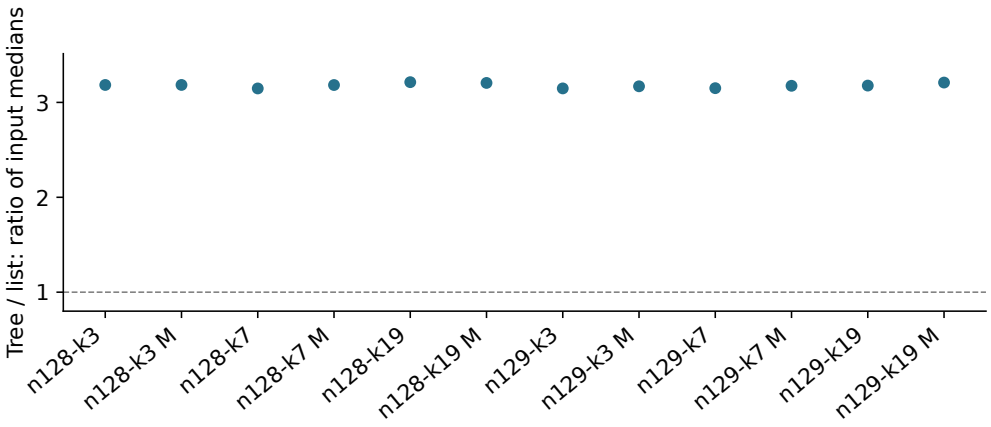


Figure 6.2.: All targeted-set input ratios. M denotes the mirrored construction. Each point is a ratio of twenty-batch medians for one input.

6.4. Variability and Measurement Accounting

For each input/backend, relative IQR is the R7-interpolated IQR divided by its own twenty-batch median. Across the common set its median is 1.37% for the list and 1.08% for the tree; maxima are 50.26% and 61.81%, respectively. The targeted-set medians are 1.18% and 1.55%, with maxima 9.08% and 7.52%. Thus typical cells have narrow central spreads, but some common-set cells do not. Every batch is retained, including the high-variability cells.

One common-set sample batch is below 10 ms; none in the targeted set is. The predetermined calibration criterion was the median of three calibration batches, not a lower bound on every later batch. The protocol therefore retained both the observation and the fixed batch size B.

Summed sample intervals are 183.511 seconds for the common set and 21.290 seconds for the targeted set. These are accumulated timed intervals, not campaign wall times: worker startup, GC preparation, validation, compression, synchronization and other orchestration occur outside them.

6.5. Supplementary Instrumentation-Hook Diagnostic

A supplementary diagnostic on October 20 uses the reviewer's fixed fifteen common-set inputs at $n = 128, 512, 513$, on the same M4 and CPython 3.12.14 in a later session. Each input has four isolated processes: list, measured tree, tree without instrumentation hooks, and additionally unwrapped operation/phase methods. Only process-local class attributes change; production files remain unchanged. Three warm-ups precede fifteen batches of 25 calls at 128 or 7 calls at 512/513. All 11,700

timed outputs and 180 warm-up outputs are saved and checked; the timer includes output append but excludes comparisons and pre-batch GC. This protocol was specific to the diagnostic.

Table 6.5.: All fifteen local diagnostic inputs. T, H and U are the measured, hook-free and additionally unwrapped trees; L is the list. Each row gives ratios of that input's batch medians. Drop is $100(1 - H/T)$, not an additive cost attribution. Original and mirror constructions are related inputs.

n	Workload	Direction	T/L	H/L	U/L	Drop (%)
128	Low	Original	3.255	1.635	1.469	49.8
128	Medium	Original	2.923	1.495	1.341	48.9
128	Heavy	Original	1.859	1.138	1.011	38.8
128	Heavy	Mirror	1.890	1.135	1.014	40.0
128	Strict-flat	Original	3.512	1.684	1.505	52.1
512	Low	Original	2.600	1.298	1.153	50.1
512	Medium	Original	2.109	1.107	0.964	47.5
512	Heavy	Original	1.112	0.689	0.623	38.0
512	Heavy	Mirror	1.118	0.693	0.612	38.0
512	Strict-flat	Original	3.086	1.487	1.305	51.8
513	Low	Original	2.556	1.262	1.123	50.6
513	Medium	Original	2.102	1.088	0.977	48.3
513	Heavy	Original	1.103	0.687	0.615	37.7
513	Heavy	Mirror	1.115	0.689	0.616	38.2
513	Strict-flat	Original	3.073	1.485	1.293	51.7

Removing instrumentation hooks lowers the tree median by 37.7–52.1% across the fifteen cases. Four of fifteen cases, the large heavy constructions, favor the hook-free tree; the remaining eleven take 1.09–1.68 times the list time. Additionally removing operation and phase wrappers (U) yields lower medians than the list in six of fifteen cases, including the two large medium-nesting cases. The diagnostic identifies a runtime effect of the configuration changes on these inputs. The qualification status, process order and environment limits are discussed in section 7.4.

6.6. Historical Rollback-Enabled Evidence

The earlier rollback-enabled study contains 3,600 measured calls on 60 inputs at its own source and environment. Python sort had the smallest median call time at every historical size. Its paper/reference ratios and counter correlations compare different pipeline scopes, not the two current backends. Full tables remain in the unchanged archive indexed in section B.1; none enters the current numerical tables.

6.7. What the Comparison Shows

On the predetermined inputs, the measured tree configuration has higher per-input median runtimes despite its stronger source-level amortized bound. Removing instrumentation hooks substantially lowers runtime in the supplementary diagnostic, although most diagnostic inputs still favor the list. These observations make the cost of the concrete Python implementation central to the comparison: field access, wrappers, local checks, allocation and tree maintenance all contribute to the measured calls. Their effects on broader workloads remain a question for further experiments.

7. Limitations

7.1. Scope and Version

The current results describe the ordinary-list and finger-tree minimal cores in the measured version, with Python sources unchanged from calibration (chapter B). The earlier rollback-enabled experiment used different code and timing scopes, so comparison with it cannot isolate the effect of removing rollback. In the current implementation, an update failure may leave a partial state that must be discarded; the invariants apply to completed updates.

7.2. Mathematical Model and Backend Fidelity

The physical and locality arguments have passed this project's source-level manual review. They have not been machine formalized or independently certified by a separate external reviewer. The endpoint and first-point rules are executable interpretations of details left implicit in the paper. The finite-parent locality case is an argument developed for this implementation; the published lemma covers the latest-child case.

The tree's linear total bound uses unit-cost comparisons and machine-word operations with expected dictionary lookup and amortized growth. Arbitrary Python comparison methods, large-number bit costs, interpreter scheduling and wall-clock behavior are outside it. Complete state audits and replay are also outside the minimal complexity claim. Changes to the implementation require reassessing the source-level argument as well as rerunning affected tests.

Ordinary lists do not inherit that bound: insertion copying alone is quadratic on increasing inputs, and their membership, ancestry and ownership work differs. The runtime comparison includes these differences in implementation. Timing curves alone establish neither complexity statement.

7.3. Input Coverage and Scale

The common set contains fourteen controlled inputs at each of ten lengths 32/33 through 512/513. Its nesting strata have fixed quotas, and the inputs are constructed rather than sampled from a population of Jordan sequences. Mirror pairs and related constructors are dependent. Consequently, the overall ratio is a descriptive median for this fixed set, not a population performance estimate.

All 3,788 recorded common-set splits per backend have an empty side, so that set does not cover the interior split path. The separate targeted set contains twelve constructions at 128/129, with nonempty smaller sides 3, 7 and 19. Its source size is always 61, its owner is finite, and its later direct updates occur only on the retained side. It does not establish broad coverage of all split ratios, black-height differences or both-output update histories. Earlier operation tests, the 18-input structural campaign and the bounded seeded supplement cover other transitions (section 4.4.1). They provide correctness evidence, not additional formal performance samples.

The narrowing common-set gap at larger sizes does not establish a crossover beyond 513 or the same trend for another generator. Constants in the implementation can dominate throughout the measured range.

7.4. Timing Scopes and Environment

The current interval includes fresh initialization, the shared algorithm, output recovery and batching overhead. Both backends receive the same scope, but their local checks, field-access wrappers and allocations differ. Their individual contributions were not isolated by the formal protocol. Disabling ledger collection retains field-access interception and wrapper dispatch. The observed slowdown therefore applies to the measured configuration. Expected-output comparison, pre-batch GC collection and evidence persistence are outside timing, while GC remains enabled during sorting.

The supplementary diagnostic in section 6.5 removes instrumentation hooks only in isolated worker processes and checks every saved output. It quantifies a configuration effect on fifteen selected inputs, not an exact additive fraction of abstract tree cost. Its inputs were selected after review of the formal results rather than fixed by the formal protocol. Variant batches are not interleaved, and process order and thermal effects are not fully controlled. Hook-free variants have not passed the full qualification protocol and are not registered production backends. The original supplied logs have weaker execution bindings (section B.1).

The formal campaigns record one Apple M4 environment with CPython 3.12.14 and battery power. Warm-ups and alternating first positions reduce selected startup and order effects, but do not

establish stable frequency, temperature or absence of all unrelated operating-system activity. The local diagnostic uses the same machine and interpreter version in a later session; it does not replicate the formal protocol.

Historical paper/reference ratios have a further scope mismatch: the reference includes certification and its oracle-backed pipeline, while the old paper call is pre-certified minimal sorting. Those ratios are not equivalent end-to-end speedups. Historical CPython 3.12.4 observations, the development pilot and current formal values are not pooled.

7.5. Statistical and Exploratory Reach

Twenty batches characterize repeat variability for each fixed input/backend. Dividing by B gives a per-call duration, not B independent samples. The median and R7 IQR describe those observations; they do not remove persistent bias or provide an independence justification for confidence intervals over related inputs. Cross-input quartiles and ranges are distributions of case summaries, not uncertainty intervals for a universal effect.

Some common-set cells have relative IQR above 50%. They remain included, without a post hoc noise filter, retry or parameter change. One sample batch falls below the calibration target, which applied to three-batch calibration medians rather than every future sample. The exact cause of either observation was not measured. The fixed batch-size table was used throughout.

Old within-size counter correlations are descriptive, based on an unbalanced mix of twelve cases per size. Checked counters and minimal times also come from different observation policies. These correlations cannot causally explain the current tree/list difference. In particular, a constant invariant counter produces an undefined correlation rather than a zero effect.

7.6. Output Provenance and Recognition

External certification and expected sorting support qualification and output checking for the paper cores. The reference pipeline instead returns oracle-derived output, as described in section 4.1.

All formal inputs are pre-certified valid. The evidence does not establish recognition accuracy, invalid-input failure reasons or recognition runtime. The sorting proof likewise starts from its validity precondition. Error correction and geometric applications remain outside this evaluation.

7.7. Consequences for the Conclusions

The cost argument and the experiments answer different questions. The former accounts for source-level operations under the stated model; the latter compares runtime on two input sets fixed in advance in one environment. Extending the performance conclusions requires broader input histories, qualified hook-free variants and measurements in additional environments.

8. Conclusion

This thesis reconstructs Simplified Jordan Sorting as one executable core with ordinary-list and heterogeneous finger-tree backends. Both recover the sorted output from their maintained state. The implementation connects the paper's operations to a source-level cost argument and a reproducible runtime comparison.

8.1. Answers to the Research Questions

RQ1: reconstruction and correctness. The shared Step 1/2/3 loop maintains geometric ownership, exact sibling groups and sorted prefixes. The argument in chapter A explains the two odd-stage z_1 corrections, temporary transfer, reversed ribs and recursive repair transients. Stable pair identities and child groups survive splitting. The two predetermined input sets passed qualification; all 66,560 formal sample calls and 1,520 warm-ups returned correct outputs.

RQ2: observed runtime. For the measured configuration retaining instrumentation hooks, every input has a higher tree median runtime. Common-set tree/list ratios have median 2.898 and range 1.114–3.535; the targeted set has median 3.180 and range 3.147–3.213. These are ratios of twenty-batch input medians for the measured implementations, including their wrapping, checking and allocation costs. A separate fifteen-input supplementary diagnostic reduces tree medians by 37.7–52.1% when instrumentation hooks are removed. Four of fifteen cases, the large heavy constructions, favor the hook-free tree; the remaining eleven take 1.09–1.68 times the list time.

RQ3: structure and cost. One potential pays propagation across mixed updates, giving amortized constant endpoint insertion and smaller-side logarithmic split. A list-size potential then pays the split terms over the full run, including bounded ownership and shared-core work. The linear total uses unit-cost comparisons and machine-word operations, expected dictionary lookup and amortized growth. Ordinary-list insertion copying alone has a quadratic lower bound.

The common workload has no two-sided splits; the targeted group contains one per input with source size 61 and smaller sides 3/7/19. The measurements thus cover different operation mixes within

complete sorting calls. The practical slowdown of the measured Python configuration is compatible with the structural bound.

8.2. Contributions and Further Work

The thesis provides an implementation whose operations can be traced to the reconstruction and cost argument, together with reproducible measurements. Independent review of the finite-parent locality extension and mixed-operation proof would strengthen confidence in the argument. The next empirical step is to qualify a production hook-free configuration and test broader split histories and environments under predeclared rules. The supplementary diagnostic and 40-input seeded correctness supplement provide starting points, while the existing data remain available for comparison. Recognition, invalid-input behavior, error correction and geometric applications would each require their own specification and evaluation.

A. Representation, Locality and Cost Argument

This appendix develops the manual source-level argument for the measured backend. The correspondence between proof steps and source code is indexed in chapter B. The geometric precondition is a simple curve with distinct, totally ordered intersections. The final cost statement uses unit-cost comparisons, machine-word operations and constant-size allocation, with expected dictionary access and amortized growth. Checked audits, replay and input certification are excluded.

A.1. Decoding and Preservation

Definition A.1 (Complete representation D)**.** Decode each header as an ordered external red–black tree. External leaves are black with rank zero and span one. An internal separator is the maximum left key; its children have equal rank, and its rank is that common rank plus one exactly when it is black. No red node has a red child, and the complete root is black. Internal span means full leaf count in an interior node, inward-branch count on a rib, and zero at the root. Header size is the total leaf count, including for a singleton.

On a left rib the left slot points up; on a right rib the right slot points up. The other slot points inward. Decoding reverses these outer chains; off-rib children already point down. Internal-root slots are empty, and header top caches identify its two logical children. Fingers identify extreme leaves; decoding is acyclic and reaches every node once. Stable leaf/handle identities bind pair records. Only endpoints carry boundary-header references. Each live header has one owner and occurs in that owner's ordered slots, of which there are at most two.

Lemma A.2 (Conversion and endpoint insertion)**.** *Conversion between a downward component and its finger representation preserves its decoded leaf sequence and identities. Singleton creation and legal endpoint insertion restore D without scanning inward subtrees.*

Proof. When exposing a rib, save its outward child before replacing that slot by the upward link. The processed chain and untouched remainder still decode to the original tree; each new rib span is the

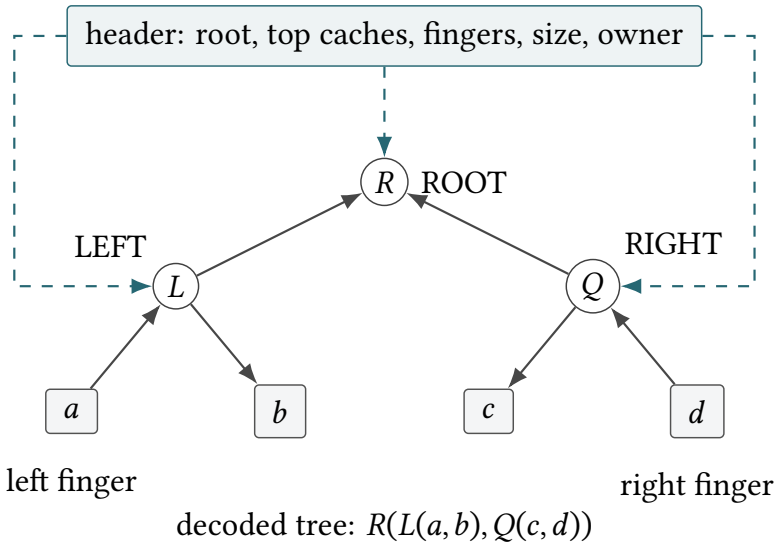


Figure A.1.: Physical decoding: solid arrows are node links; dashed arrows are header references. Outer chains point up and inward branches down; root slots are empty. Header fingers reference leaves a, d (arrows omitted). Colours and sizes are omitted.

unchanged inward count. The inverse saves the upward link, attaches the already processed lower component and refreshes rank and full span bottom-up. A leaf needs no conversion; a nonroot top exposes one rib, an internal root two. Final root blackening may increase rank by one.

Insertion attaches a black leaf and red connector at the selected finger. Only the connector's upper edge can violate red adjacency. A red uncle is blackened with the parent while the grandparent becomes red; the latter's rank is unchanged and the conflict moves upward. With a black uncle the left-side rotation is $G(P(C, M), F) \mapsto P(C, G(M, F))$. Initially P, C are red and G, F black. The three blocks C, M, F have equal rank. Afterwards G is red, P black, and the total rank is unchanged. Separators remain $\max M$ at G and $\max C$ at P.

For nonroot G, it becomes interior with children M, F; P retains its rib role, points to the former ancestor and has inward child G. For root G, P becomes the empty-slot root, G becomes the opposite rib with inward child M, and top caches become C, G. The unchanged upward links from C and F already name their correct parents. Spans follow these roles. The mirror is $G(F, P(M, C)) \mapsto P(G(F, M), C)$, with separators $\max F$ and $\max M$, still maxima of left blocks. Black-parent termination, rotation or root blackening restores D. Only the replaced endpoint's boundary slot changes; the live list ID is retained. □

Lemma A.3 (Recursive downward join contract)**.** *Joining two disjoint ordered components preserves*

their leaf union and order. The recursive return has equal child black heights and correct metadata, but may contain one root–child red edge when its root is red. A complete nonempty join restores the red–black invariants.

Proof. Both input roots are blackened. Descent follows the right path of a taller left component, or the left path of a taller right component; the other argument remains the original blackened root. A black step lowers rank by one and a red step preserves it, so descent reaches equal-rank black roots without crossing their rank or changing sides. There a red connector forms an internally valid base case with the same rank. On return, the replaced component keeps that rank. A black parent without conflict is refreshed; only an outer RR conflict on right descent, or LL on left descent, can arise. One rotation produces a red middle root with two black children and unchanged black height. The LR/RL helper branches are defensive: direct helper fixtures exercise them, but valid recursive joins do not reach them.

A red parent takes the direct-return branch. Its originally black updated child can return an internally valid red root; the other child stays black. Consequently only the updated root–child edge may be red–red. This is a recursive transient, not D. The original parent of that red node is black and repairs this LL or RR conflict on the next return. The induction excludes a deeper or second simultaneous red conflict. Descent never searches through a return transient.

At the top, either the valid base case or a black-root repair returns an internally valid tree. Final root blackening changes only root colour; it cannot repair an internal red edge. For example, joining $A_B([0], P_R([1], [2]))$ to $[3]$ temporarily returns $P_R([1], C_R([2], [3]))$, then repairs at A_B; the mirror uses LL. In a left rotation $A(X, B(Y, Z)) \mapsto B(A(X, Y), Z)$, separators stay $\max X, \max Y$; refreshing bottom-up gives full spans and ranks. The connector separator is the left input's maximum and sizes add. An empty-input branch simply returns the other component and retires the unused connector; this branch alone need not blacken its root. □

Lemma A.4 (Detachment and split preservation)**.** *Legal split and adoption preserve the leaf set, stable identities and descendant groups, and restore D and header ownership at their completion.*

Proof. Detachment removes an extreme component and its parent connector, replacing the latter by its sibling. A red parent loses no black height; a red sibling replacing a black parent is blackened. Otherwise the replacement is short by one black layer. Use a transient D^*: the short component's rank is actual, but ancestors still account for one virtual black layer on that path. Refreshing these ancestors as ordinary equal-height trees would be invalid.

For short rank r, the sibling has rank $r + 1$, so is not a leaf. A red sibling is blackened and the black parent reddened before rotation, reducing to a black sibling. If both nephews are black, redden the

sibling: a red parent absorbs the deficit by blackening; a black parent decreases rank and becomes the next short component. A red near nephew with black far nephew is rotated and recoloured into the far-red case. In that case the sibling takes the parent colour, the parent and far nephew become black, and a rotation absorbs the deficit. A deficit reaching the root lowers the whole black height by one. These are all repair branches.

For a left deficit the outer rotation is $P(S, B(N, F)) \mapsto B(P(S, N), F)$, with separators $\max S, \max N$. P is a left rib pointing up to B and inward to N, with span $|N|$. At the root, B gets empty slots, span zero and top caches P, F; otherwise it retains the rib with inward F and the old upward link. The mirror keeps separators $\max F, \max N$ in $P(B(F, N), S) \mapsto B(F, P(N, S))$. A near rotation at the opposite root rib changes $B(N(X, Y), F)$ to $N(X, B(Y, F))$: the inward spans become $|X|, |Y|$, and the appropriate top cache becomes N. The far upward link is unchanged. These substitutions preserve order and update only affected metadata.

The selected component need not straddle the cut: it can lie wholly on one side. The remainder lies wholly on a known output side. Downward splitting classifies leaves by key $\leq v$ or $> v$; each saved sibling is joined once to the corresponding accumulator in order, using lemma A.3. Connectors are reused once or retired, never leaves. Finger joins stop at equal black rank and use lemma A.2; conversion handles their component boundaries. Size starts at one at a finger and accumulates inward spans on ascent; at the root use header size. The remainder is obtained by subtraction, downward sizes by full spans, and joined sizes by addition. There is no output-wide counting pass.

Every split retires the source header, including an empty-side fast return, and allocates fresh IDs for nonempty outputs. It clears old endpoint references and sets new ones. Adoption checks the pending source, owner, new parent and output IDs, then changes headers and at most two child slots. Leaves, handles and their own child groups are unchanged. Strict containment of extreme children implies containment of the whole ordered group. In the sorting call the new parent was childless, so acquiring its strictly contained old children cannot create a cycle. Pending overlap is allowed only for this planned transfer; completion clears it before output insertion. □

A.2. Locality and Exact Grouping

Use G from section 3.2.1, not merely a bound of two groups. An owner is the smallest strictly enclosing same-parity pair, or the parity dummy. Its region R is bounded by its arc and its direct children's arcs, with intervening open axis gaps. Descendant arcs lie behind child boundaries, not inside R.

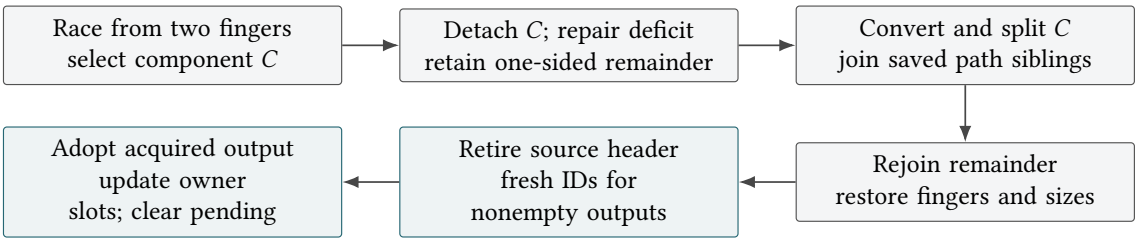


Figure A.2.: Two-sided split and adoption. The selected component may itself yield an empty side. Sizes come from path spans and header totals; transfer changes header ownership rather than rebinding every leaf.

Lemma A.5 (Applicable re-entry gaps)**.** *Suppose the region has not changed since its latest member P_j was added. If P_j is a child, first re-entry from z_j uses its adjacent region gap, or, for the dummy, the opposite unbounded gap. In the latter case, z_j and the re-entry point z_k bracket all other finite endpoints of the region. If P_j is the finite parent, first re-entry uses the gap immediately inward from z_j.*

Proof. The child case is the locality lemma of Fung et al. [2, p. 88, locality lemma (a)–(b), Figs. 3–4], under its latest-child and unchanged-region hypotheses; part (b) supplies the dummy bracketing condition. It is not applied to a latest finite parent. For that additional case let q be the first child endpoint inward from z_j, or the other parent endpoint if there is no child.

Assume the first re-entry z_k is outside (z_j, q). Choose a child endpoint a strictly between z_j and z_k along that boundary, and let $b = z_{j-1}$ be the other parent endpoint. An auxiliary simple crosscut inside R connects z_j to z_k and separates these boundary vertices. The actual curve portion from z_j to z_k lies outside R by first re-entry and meets the crosscut only at its ends. Together they form a simple closed curve separating a and b. The old curve portion connecting a to b cannot cross the crosscut, because the old curve avoids the region's interior; it cannot meet its newer endpoints or cross the actual later portion, by simplicity. This contradicts separation. If there is no child, the entire inward gap is already (z_j, q). Unlike the dummy, a finite parent has no second unbounded outside gap. This establishes the extra case. □

Lemma A.6 (Legal calls and prefix induction)**.** *Starting from initialization, G, geometric ownership and the sorted-prefix invariant imply legal endpoint insertion, same-owner/different-group transfer when required, and correct output insertion. Each completed iteration restores these invariants.*

Proof. Initialization has one singleton per parity family and correctly orders three points. Consider iteration i, with $x = z_{i-1}$ and new point $y = z_i$. Let p be the smallest same-parity interval containing x, or the dummy, and let P_j be the latest member of its owner region. Updates inside older descendants

have not changed that region. Were there an earlier first re-entry, its next same-parity arc would create a new direct child of p, possibly acquiring consecutive children. Its end index would exceed j, contradicting the definition of j. Here alternation between the two parities is essential. Thus lemma A.5 applies to x.

Except for x and, at odd i, z_1, processed points are endpoints of same-parity pairs. Step 1/2 skips this exceptional z_1. Parent and direct child endpoints block all more distant ancestors and descendants in the neighbor search. Consequently an interior entry gap gives the last endpoint of the left group and first endpoint of the right group; an outer gap gives one group boundary and the parent. By G these are designated endpoints, not arbitrary members. The two groups, when both present, have the same owner and different live IDs.

For $x < y$, simplicity implies that y remains inside finite p and is neither inside a direct child's interval nor equal to its endpoints. If the left boundary is p, Step 3(a) creates a singleton in a free slot. In the dummy outer-gap case, the bracketing condition leaves the group beyond z_j empty, so the owner has at most one group. When the left boundary is a child, Step 3(a) instead appends to the left group. If the right boundary is p, there are no children to transfer. Otherwise its right group is the source. Reading that source before insertion is safe: Step 3(a) changes only the other group or a new slot, preserving source identity and owner.

All source children lie to the right of x. Since y cannot lie inside a child, a source key $\ell_h \le y$ is equivalent to $x < \ell_h < r_h < y$. Hence value splitting acquires precisely the consecutive children enclosed by the new pair. The temporary overlap is exactly the declared transfer. After adoption, p's latest child is P_i and its groups are separated by y; P_i is the latest parent of its own single acquired group. Other owners' groups, descendant identities and the opposite parity are unchanged. Strict enclosure gives acyclicity.

Among old paired endpoints between x and y, the largest is the high endpoint of the rightmost acquired direct child, or x if none exists. Descendants cannot exceed that child's extreme. This is the predecessor ignoring the exceptional odd-stage z_1; replacing the anchor by z_1 exactly when it lies between this base and y gives the true predecessor. The decreasing case reflects the argument, using the low endpoint and successor. Step 3(c) therefore preserves sorted order after the pending-state fence. This proves the simultaneous prefix induction. □

A.3. Costs of Mixed Operations

Lemma A.7 (Cumulative join cost)**.** *Splitting a downward component of raw rank r costs $O(r+1)$ in joins and path work. It does not require $O(r)$ joins each independently charged $O(r)$.*

Proof. Write v for rank after blackening the root. For nonempty join inputs of normalized heights a, b, the result has height c and descent count Q with

$$\max(a,b) \le c \le \max(a,b)+1, \qquad Q \le 2|a-b|+3.$$

Descent follows a legal tree: each black level decreases the height difference and there cannot be consecutive red nodes. Each return repair is constant work by lemma A.3. An increase in height leaves a black root with black children: before final blackening it was the valid red base or a black-root repair result, not the red-root transient. Pre-blackening an originally red input adds at most one separately.

Inductively, splitting a legal raw-rank-r component gives nonempty outputs of normalized height at most $r+1$. For a black input attaining $r+1$, the output root and children are black. For the induction step let child rank be q. At a black root, both join inputs have normalized height at most $q+1$, so the output is at most $q+2 = r+1$, with the stated strong case. At a red root the children are black. If the recursive output reaches $q+1$, its root and children are black by that strong case; joining a height-q sibling descends one level and cannot increase height again. Otherwise the ordinary join bound suffices.

Thus at each backtracking join, accumulated height a and untouched sibling height b satisfy $a-b \le 1$. The displayed bounds imply $Q \le 2(c-a)+5$. Each of the two output accumulators has nondecreasing normalized height, bounded by $r+1$. Summing over $J = O(r+1)$ joins gives $\sum Q \le 2(v_L+v_R)+5J = O(r+1)$. Empty-side returns add only constant work per frame. This pays descent, reconstruction and metadata maintenance. □

Lemma A.8 (Finger distance and opposite-rib work)**.** *For smaller nonempty output size s, the selected component has rank $r = O(\log(s+1))$. Opposite-rib access in detachment costs $O(r+D+1)$, where D counts continuing deletion propagations.*

Proof. The left search climbs only while the parent separator is below the cut; the right search has the reflected condition. It is the component *before* a climb that is known to be wholly on one side, not necessarily its parent. A one-sided component of rank t has at least 2^t leaves. On the smaller side this is at most s, and one climb increases rank by at most one. Alternating searches differ by at most one step; no two red levels are consecutive. Hence both paths, and the selected component, have the stated rank bound. If $s = 0$, the endpoint fast path costs $O(1)$.

An inward sibling is interior except at the root. Only there can `rib_child` require an opposite-rib walk. A red-sibling root rotation makes the continuing parent nonroot; propagation at the root terminates. A near rotation permits at most one additional far-nephew lookup. There are therefore at most two

such walks per repair. With initial short rank $r_0 \le r + O(1)$, each continuing propagation raises actual rank by one; the reached root has rank at most $r_0 + D + 2$. This proves the walk bound. Conversion visits $O(r + 1)$ levels; finger joining climbs the large component only to the small component's rank, also $O(r + 1)$. Any ensuing insertion propagation is charged next. □

Lemma A.9 (One potential across mixed updates)**.** *Endpoint insertion costs amortized $O(1)$, and split costs $O(\log(s + 2))$, including local repairs, conversion and size maintenance. Creation and adoption have constant cost.*

Proof. On decoded logical internal nodes define

$$\Phi = \#\{\text{black nodes with two black children}\} + 2\#\{\text{black nodes with two red children}\}.$$

Count each node once across the live forest and temporarily disjoint components; never reset the potential when headers change. Leaves contribute zero. An insertion propagation changes a black/red-red grandparent of weight two into a red node of weight zero; the blackened parent is mixed and the blackened uncle has two black children, contributing one. The decrease is one per continuing propagation. An upper boundary or terminal repair changes only constantly many weights.

In a continuing deletion propagation, the black sibling with black children becomes red, losing one, and the black parent changes from two black children to mixed, losing another. The next ancestor's child stays black. Thus each such step decreases Φ by two; absorption, rotations and root treatment change constantly many weights. Direction conversion alone changes neither logical children nor colours. The $O(r + 1)$ join/path steps each affect a constant number of weights, and any finger-join propagation uses this same potential. For constants independent of the sequence,

$$\Delta\Phi \le C(r + 1) - D - I, \qquad W \le C'(r + D + I + 1),$$

where I counts propagation rounds, not individual recolourings. Choose a sufficiently large constant multiplier of Φ to pay both propagation and the D term in the opposite-rib work. Lemmas A.7 and A.8 then give the split bound. The same accounting with constant boundary work gives endpoint insertion. Adoption touches bounded header/owner slots, not a leafwise ownership map. □

Theorem A.10 (Total valid-input work)**.** *Under the stated computation model, the shared minimal core with this finger backend uses $O(n)$ expected/amortized work and $O(n)$ space on a valid input of length n.*

Proof. By lemma A.6, each iteration performs constantly many legal boundary accesses and at most one creation/insertion, split and adoption. To pay the split terms use the nonnegative list potential $\Psi = c \sum_{L \neq \emptyset} (|L| - \log_2 |L|)$. Creation increases it by c; insertion into size m increases it by $c(1 - \log_2(1 + 1/m)) \leq c$; adoption changes no sizes. For a nonempty split $m = a + b$ with $1 \leq a \leq b$,

$$\Delta\Psi = c \log_2 \frac{a+b}{ab} \leq c(1 - \log_2 a).$$

Since $\log_2(a+2) \leq \log_2 a + \log_2 3$, a sufficiently large c pays the smaller-side term. An empty-side split has $\Delta\Psi = 0$ and constant amortized cost. Combine this with the single multiple of Φ in lemma A.9; initialization has constant potential.

The remaining shared-core work consists of bounded endpoint/handle and owner-slot accesses, geometric comparisons, stage records and linked output insertion per iteration. It performs no membership or ancestor scan in this backend. Input materialization and final output traversal are linear. Access wrappers and iterator resumption add constant work per affected access, not zero work. Constant-size records per iteration and linear node/handle registries give the space bound, including retired-header bookkeeping. Nonnegative final potentials complete the aggregate bound. □

The theorem accounts for the shared minimal core and its tree operations in the model stated at the start of this appendix. The measured runtimes are reported in chapter 6.

B. Evidence and AI Use

B.1. Evidence and Source Index

Current experiments. The measured version is `4a83fa6` (5,600 common-set and 480 targeted-set batches). Its Python sources are identical to those of the calibration version, `c5b5ac3`. Qualification sources are `3cafb44` for the 140-input common set and `668371f` for the 12-input split set.

Development evidence. Source identities for table 4.1 are `cc1aa2d` (operation tests), `84f7e13` (structural campaign) and `19a553fb` (seeded supplement, October 20 protocol). The core access interfaces were introduced at `71e3e71`.

The archive index and proof-to-source map cover development, historical and supplementary evidence: `docs/thesis/evidence_source_index_20261022.md`.

B.2. Documentation of AI Use

The available record documents Codex desktop (OpenAI) revision activity on October 11–22, 2026; earlier use is incompletely recorded. Claude (Anthropic) reviews are reported on October 17/20, with discussion on October 22. ChatGPT web (OpenAI) supported translation, algorithm explanations, review discussions and planning; exact dates and passages are not recoverable from the available record. Model versions are also incompletely recorded.

Recorded Codex assistance includes technical changes and checks, manuscript restructuring and revised formulations in the abstract and Chapters 1, 3–8, English proof exposition in Appendix A, and Appendix B's disclosure/reflection. Claude's review led to the fifteen-row diagnostic table and four/eleven qualification, and the dummy bracketing premise in lemma A.5; Codex assisted with both revisions. This identifies affected material, not whole-chapter authorship. A passage/type map, request/output examples and the proposed local-marking scheme are in `docs/development/AI_USE_RECORD_20261022_scope.md`. The scheme awaits supervisor confirmation, and the disclosure remains incomplete. A separate signed declaration is required.

On October 31, 2026, Codex used the Academic Humanizer skill to revise the language of the abstract, Chapters 1–8 and Appendices A–B. This pass reduced repeated scope qualifications and improved transitions. Its change record is `docs/thesis/language_revision_20261031.md`. A further revision on the same day restructured the abstract, clarified the research questions and terminology, and prepared the review copy dated November 2. The changes and checks are recorded in `docs/thesis/supervisor_revision_20261102.md`.

B.3. Reflection on AI Use

My contribution centred on defining the research requirements and directing the project's development and evaluation. I made the efficient backend a core requirement and kept development focused on it. I also required the implementation to recover its output from its own maintained state. When assessing proposed changes and presenting results, I distinguished implemented functionality, bounded testing, complexity arguments, and measured performance.

I used Codex, Claude, and ChatGPT for technical and language assistance. They contributed implementation suggestions, code changes, proof formulations, and review findings. I communicated requirements, set priorities, requested explanations, and decided which revisions to pursue. When a data structure was unfamiliar, I asked for examples explaining the relationships between the Jordan algorithm, sibling groups, and the red–black tree. Particularly useful assistance included finding literature, discussing the project architecture, expressing ideas academically, and checking LaTeX syntax. I used ChatGPT's web interface for translation, algorithm explanations, and discussions of review findings and project planning.

Claude-assisted review identified a missing premise in the English proof exposition and an incomplete diagnostic table. I included both findings in the revision process and distinguished corrections to the exposition from changes to the implementation. The measured tree configuration had higher per-input median runtimes than the list backend throughout the formal comparison. I accepted this outcome and required the results and their limitations to be reported accurately.

I cannot identify a specific task in this project that I believe would have been faster without AI. Checking suggestions, understanding unfamiliar concepts, and revising incomplete answers required effort. The omitted proof premise showed me the need to check the reasoning behind fluent explanations. I also had to assess successful tests in relation to the cases they covered and examine general correctness and complexity arguments separately.

I am satisfied with the outcome. AI assistance helped me organize the work, explore technical questions, and express ideas more clearly. The experience reinforced the importance of clear questions,

explicit acceptance criteria, and sufficient understanding to assess proposed changes. In future work, I would document my decisions and the reasons for accepting or rejecting suggestions more systematically, alongside the AI contributions themselves. This would make both my reasoning and the role of the tools easier to trace. Responsibility for understanding and defending the submitted work remains with me.

Language assistance: this reflection was revised with Codex from the author's supplied account and English draft.

Bibliography

[1] K. Hoffmann, K. Mehlhorn, P. Rosenstiehl, and R. E. Tarjan. "Sorting Jordan Sequences in Linear Time Using Level-Linked Search Trees". In: *Information and Control* 68.1–3 (1986), pp. 170–184. DOI: 10.1016/S0019-9958(86)80033-X.

[2] K. Y. Fung, T. M. Nicholl, R. E. Tarjan, and C. J. Van Wyk. "Simplified Linear-Time Jordan Sorting and Polygon Clipping". In: *Information Processing Letters* 35.2 (June 1990), pp. 85–92. DOI: 10.1016/0020-0190(90)90111-A.

[3] P. Sanders. "Algorithm Engineering: An Attempt at a Definition". In: *Efficient Algorithms: Essays Dedicated to Kurt Mehlhorn on the Occasion of His 60th Birthday*. Ed. by S. Albers, H. Alt, and S. Näher. Vol. 5760. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2009, pp. 321–340. DOI: 10.1007/978-3-642-03456-5_22.

[4] C. C. McGeoch. *A Guide to Experimental Algorithmics*. Cambridge: Cambridge University Press, 2012. DOI: 10.1017/CBO9780511843747.

[5] R. E. Tarjan and C. J. Van Wyk. "An $O(n \log \log n)$-Time Algorithm for Triangulating a Simple Polygon". In: *SIAM Journal on Computing* 17.1 (Feb. 1988), pp. 143–178. DOI: 10.1137/0217010.

[6] M. R. Brown and R. E. Tarjan. "A Representation for Linear Lists with Movable Fingers". In: *Proceedings of the Tenth Annual ACM Symposium on Theory of Computing*. STOC '78. New York, NY, USA: Association for Computing Machinery, 1978, pp. 19–29. DOI: 10.1145/800133.804328.

[7] M. R. Brown and R. E. Tarjan. "Design and Analysis of a Data Structure for Representing Sorted Lists". In: *SIAM Journal on Computing* 9.3 (Aug. 1980), pp. 594–614. DOI: 10.1137/0209045.

[8] S. Huddleston and K. Mehlhorn. "A New Data Structure for Representing Sorted Lists". In: *Acta Informatica* 17.2 (1982), pp. 157–184. DOI: 10.1007/BF00288968.

[9] K. Hoffmann, K. Mehlhorn, P. Rosenstiehl, and R. E. Tarjan. "Sorting Jordan Sequences in Linear Time". In: *Proceedings of the First Annual Symposium on Computational Geometry*. SCG '85. New York, NY, USA: Association for Computing Machinery, 1985, pp. 196–203. DOI: 10.1145/323233.323259.

[10] R. Bar-Yehuda and B. Chazelle. "Triangulating Disjoint Jordan Chains". In: *International Journal of Computational Geometry & Applications* 4.4 (1994), pp. 475–481. DOI: 10.1142/S0218195994000252.

[11] F. Aurenhammer. “Jordan Sorting via Convex Hulls of Certain Non-Simple Polygons”. In: *Proceedings of the Third Annual Symposium on Computational Geometry*. SCG ’87. New York, NY, USA: Association for Computing Machinery, 1987, pp. 21–29. DOI: 10.1145/41958.41961.

[12] E. Sojka. “Two Simple and Efficient Algorithms for Jordan Sorting and Polygon Cutting and Clipping”. In: *Computer Networks and ISDN Systems* 29.14 (Oct. 1997), pp. 1661–1673. DOI: 10.1016/S0169-7552(97)00081-0.

[13] E. Sojka. “A New Algorithm for Jordan Sorting: Its Average-Case Analysis”. In: *Discrete Applied Mathematics* 93.1 (Apr. 1999), pp. 127–140. DOI: 10.1016/S0166-218X(99)00011-6.

[14] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney. “Producing Wrong Data Without Doing Anything Obviously Wrong!” In: *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS XIV. New York, NY, USA: Association for Computing Machinery, 2009, pp. 265–276. DOI: 10.1145/1508244.1508275.

[15] T. Kalibera and R. E. Jones. “Rigorous Benchmarking in Reasonable Time”. In: *Proceedings of the 2013 International Symposium on Memory Management*. ISMM ’13. New York, NY, USA: Association for Computing Machinery, 2013, pp. 63–74. DOI: 10.1145/2464157.2464160.